
MEMOIRE

Présenté à

Institut Supérieur d'Informatique de Mahdia

Effectué à

Digi Growing

En vue de l'obtention du diplôme de

Licence Computer Engineering : Systèmes Embarqués et Internet des Objets
Licence Computer Engineering : Systèmes Embarqués et Internet des Objets

Elaboré par :

AYMEN MABROUK ,BEN RAYANA YASSINE

Assistant Robotisé IA pour Ordinateurs de Bureau

*Soutenu le : 13 juin 2025
devant le jury composé de :*

Président	: ZAHRA RADDAOUI
Rapporteur	: NADER FNAIECH
Encadreur Académique	: BEHIJ BEN AMOR
Encadrant Professionnel	: CHAWKI GHARSALOU

Dédicaces

Je consacre ce modeste projet

À mes chers parents, **Nouri** et **Sihem**, pour leur amour inconditionnel, leur soutien indéfectible, leurs sacrifices silencieux et leur confiance en moi. Ce travail est le fruit de vos encouragements constants et de votre présence rassurante. Je vous en suis profondément reconnaissant.

À mes frères, pour votre soutien, votre bienveillance et vos mots d'encouragement dans les moments les plus difficiles. Votre présence m'a toujours donné la force d'avancer.

À mes amis d'enfance, avec qui j'ai grandi et partagé tant de souvenirs précieux. Votre amitié sincère est un trésor inestimable qui m'a accompagné tout au long de mon parcours.

À tous mes amis rencontrés au fil des années d'études, pour les moments partagés, l'entraide, les éclats de rire et le soutien mutuel. Votre présence a rendu ce chemin plus léger et plus riche.

À toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet, recevez ici l'expression de ma sincère reconnaissance.

Que ce travail soit le reflet de votre soutien inestimable et le début de nombreux accomplissements à venir. Merci d'avoir toujours cru en moi.

Ben Rayana Yassine

Dédicaces

Je consacre ce modeste projet

À mes chers parents, **Hamed** et **Nabila**, pour leur amour inconditionnel, leur soutien indéfectible, leurs sacrifices silencieux et leur confiance en moi. Ce travail est le fruit de vos encouragements constants et de votre présence rassurante. Je vous en suis profondément reconnaissant.

À mes frères, **Ali** et **Ahmed**, pour votre soutien, votre bienveillance et vos mots d'encouragement dans les moments les plus difficiles. Votre présence m'a toujours donné la force d'avancer.

À ma chère sœur, **Imen**, pour ta tendresse, ton écoute, et ton soutien affectueux tout au long de ce parcours. Merci d'avoir été là, avec ton sourire et ton énergie positive.

À mes amis d'enfance, avec qui j'ai grandi et partagé tant de souvenirs précieux. Votre amitié sincère est un trésor inestimable qui m'a accompagné tout au long de mon parcours.

À tous mes amis rencontrés au fil des années d'études, pour les moments partagés, l'entraide, les éclats de rire et le soutien mutuel. Votre présence a rendu ce chemin plus léger et plus riche.

À toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet, recevez ici l'expression de ma sincère reconnaissance.

Que ce travail soit le reflet de votre soutien inestimable et le début de nombreux accomplissements à venir. Merci d'avoir toujours cru en moi.

Aymen Mabrouk

Remerciements

À la fin de cette tâche, je souhaite exprimer ma sincère reconnaissance envers mon professeur et superviseur, **Monsieur Behij Ben Amor**, pour son suivi constant, son accompagnement bienveillant et son soutien inestimable tout au long de ce projet.

Je souhaite également exprimer ma gratitude à mon encadrant professionnel, **Monsieur Chawki Gharsellaoui**, pour ses précieux conseils, ses orientations claires, ainsi que sa motivation continue.

Je remercie sincèrement toute l'équipe pédagogique de l'Institut Supérieur d'Informatique de Mahdia, ainsi que les intervenants professionnels, pour la qualité de la formation dispensée et leur disponibilité.

Je tiens à exprimer ma profonde gratitude aux membres du jury, **Madame Zahra Raddaoui** (Présidente) et **Monsieur Nader Fnaiech** (Rapporteur), pour l'intérêt qu'ils ont porté à ce mémoire, pour le temps qu'ils y ont consacré, et pour leurs remarques constructives qui contribueront sans nul doute à l'amélioration de ce travail.

TABLE DES MATIÈRES _____

Table des matières

Introduction Générale	1
1 Contexte général du projet	3
1.1 Introduction	4
1.2 Présentation de l'entreprise	4
1.2.1 Digi Growing	4
1.2.2 Secteur d'activités	5
1.3 Cahier des charges	5
1.3.1 Besoins fonctionnels	5
1.3.2 Besoins non fonctionnels	5
1.3.3 Identification des acteurs	5
1.4 Méthode de développement	5
1.4.1 Méthode adoptée	6
1.4.2 Constitution de l'équipe	7
1.5 Étude de l'existant	7
1.6 Critique de l'existant	8
1.7 Présentation Générale	8
1.8 Objectifs de la solution	8
1.9 Architecture du système	9
1.10 Fonctionnement général	9
1.11 Avantages de la solution	9
1.12 Product Backlog	10
1.13 Organisation des Sprints	11
1.14 Détail des Sprints	11
1.14.1 Sprint 1 – Conception & Modélisation du système	11

1.14.2	Sprint 2 – Développement du système IoT	11
1.14.3	Sprint 3 – Modélisation IA et Gestion des données	12
1.14.4	Sprint 4 – Design 3D & Interface mobile	12
1.15	Critères d’acceptation des User Stories (Definition of Ready - DoR)	13
1.16	Conclusion	13
2	Sprint 1 - Conception et Modélisation du Système	14
2.1	Introduction	15
2.2	Analyse et Spécification des Besoins	15
2.2.1	Spécification des Besoins	15
2.2.2	Analyse des Besoins	15
2.2.3	Identification des Cas d’Utilisation	16
2.3	Diagramme de Cas d’Utilisation	16
2.3.1	Diagramme de Cas d’Utilisation Global	17
2.3.2	Diagramme de Cas d’Utilisation Détaillé	17
2.4	Diagramme de Classes	19
2.4.1	Introduction	19
2.4.2	Diagramme Principal	19
2.5	Diagrammes de Séquence	22
2.5.1	Diagramme de séquence principal	22
2.6	Diagramme de cas d’utilisation de l’application mobile BEEMO	23
2.6.1	Diagramme de Séquence de l’Application Mobile	27
2.7	Conclusion	29
3	Sprint - 2 : Réalisation du prototype	30
3.1	Introduction	31
3.2	Environnement logiciel	31
3.3	Environnement matériel	33
3.4	Installation et configuration des cartes	37
3.5	Architecture générale du système	39
3.6	Schéma de câblage du prototype	41
3.7	Prototypes du robot BEEMO	43
3.8	Conclusion	44
4	Sprint - 3 : Modélisation IA	45
4.1	Intelligence Artificielle (IA)	46
4.1.1	Machine Learning	46
4.1.2	Deep Learning	46
4.1.3	Computer Vision	47
4.2	Réseaux de Neurones Convolutifs (CNN)	47

4.2.1	Couche Convulsive	48
4.2.2	Architecture du traitement vocal basé sur l'IA	48
4.2.3	Analyse des métriques d'entraînement	49
4.3	Conclusion	56
5	Sprint - 4 : Design 3D & Interface mobile	57
5.1	Introduction	58
5.2	Conception 3D du prototype	58
5.2.1	Présentation de Blender	58
5.2.2	Design du robot BEEMO	59
5.2.3	Processus de conception	60
5.2.4	Composants modélisés	60
5.2.5	Caractéristiques techniques du boîtier	61
5.2.6	Optimisations pour l'impression 3D	61
5.3	Développement de l'interface mobile	62
5.3.1	Flutter : Framework de développement	62
5.3.2	Architecture de l'application	62
5.3.3	Fonctionnalités principales	63
5.3.4	Design UX/UI	63
5.3.5	Sécurisation de l'application	70
5.3.6	Gestion des appareils et des lieux	70
5.3.7	Tests et validation	70
5.4	Conclusion	71
	Bibliographie	74

TABLE DES FIGURES _____

Table des figures

1.1	Logo de Digi Growing	4
1.2	Cycle Scrum basé sur des sprints successifs	6
1.3	Organisation d'un sprint Scrum	7
1.4	Rôles dans une équipe Scrum	7
2.1	Diagramme des acteurs du système	16
2.2	Diagramme de cas d'utilisation global du système	17
2.3	Diagramme de cas d'utilisation détaillé du système	18
2.4	Diagramme de classes du système d'assistant robotisé	20
2.5	Diagramme de séquence principal du système	22
2.6	Diagramme de cas d'utilisation de l'application mobile BEEMO	24
2.7	Diagramme de classes de l'application mobile BEEMO	25
2.8	Diagramme de séquence de l'application mobile BEEMO	28
3.1	Logo UML	31
3.2	Logo Visual Studio Code	32
3.3	Logo Flutter	32
3.4	Logo Firebase	32
3.5	Logo LaTeX	32
3.6	Logo Figma	33
3.7	Logo Kaggle	33
3.8	Carte Raspberry Pi 4	33
3.9	Écran LCD 2 pouces IPS	34
3.10	Haut-parleurs 8	34
3.11	Microphone MEMS INMP441	34
3.12	Amplificateur PAM8403	35
3.13	Capteur de vibration SW-420	35

3.14	Module tactile TTP223B	35
3.15	Carte driver de servomoteurs	36
3.16	Servomoteur MG90D	36
3.17	Mini servomoteur SG90	36
3.18	Plaque perforée de prototypage	37
3.19	Convertisseur Micro USB vers 5V	37
3.20	Interface de Raspberry Pi Imager (choix de l'OS)	38
3.21	Écriture de l'image sur la carte SD avec Raspberry Pi Imager	38
3.22	Connexion à l'écran du Raspberry Pi via VNC Viewer	39
3.23	Architecture générale du système BEEMO	40
3.24	Schéma de câblage du robot BEEMO	41
3.25	Différentes vues du prototype physique du robot BEEMO.	43
4.1	Les sous-domaines de l'intelligence artificielle	46
4.2	Structure d'un réseau de neurones convolutif	47
4.3	Architecture de traitement de commande vocale	48
4.4	Progression des pertes d'entraînement et de validation au fil du temps	49
4.5	Progression du taux d'apprentissage durant l'entraînement	50
4.6	Histogramme des valeurs de perte d'entraînement et de validation	51
4.7	Consommation de mémoire RAM et GPU durant l'entraînement	52
4.8	Taux de changement des valeurs de perte entre étapes consécutives	53
4.9	Visualisation en carte thermique de la progression de la perte d'entraînement	53
4.10	Ratio entre perte d'entraînement et perte de validation au fil du temps	54
4.11	Moyennes mobiles de la perte de validation avec différentes tailles de fenêtre	55
5.1	Logo de Blender	58
5.2	Différentes vues du robot BEEMO	60
5.3	Logo de Flutter	62
5.4	Écran principal du Tableau de Bord (Dashboard)	64
5.5	Fenêtre de Connexion au Robot	65
5.6	Écran d'État et de Configuration du Robot	66
5.7	Interface : Gestion des Pièces et des Appareils ("Sala")	67
5.8	Écran de Chat/Assistant (PFE Platform Assistant)	68
5.9	Contraintes matérielles pour l'exécution locale du modèle AI tunisien.	73

*

LISTE DES TABLEAUX_____

Liste des tableaux

1.1	Backlog produit – Assistant IA robotisé connecté (User Stories)	10
1.2	Organisation des sprints du projet d’assistant connecté	11
2.1	Tableau descriptif "Identification des cas d’utilisations"	16

*

LISTE DES ABRÉVIATIONS

IA	Intelligence Artificielle
IoT	Internet of Things (Internet des Objets)
CPU	Central Processing Unit
API	Application Programming Interface
RAM	Random Access Memory
OS	Operating System
ISIMa	Institut Supérieur d'Informatique de Mahdia
NLP	Traitement du Langage Naturel
OLED	Type d'écran utilisé pour l'affichage
DoR	Definition of Ready
UML	Utilisé pour la modélisation
STT	Speech-to-Text
JWT	Token utilisé pour la connexion sécurisée
UX/UI	User Experience / User Interface
VOSK	Module de reconnaissance vocale
DeepSeek	Modèle d'IA distant mentionné
GPT	Modèle d'IA distant mentionné

Introduction Générale

À l'ère de la transformation numérique, les interactions entre l'homme et la machine sont devenues un enjeu central dans la conception des systèmes informatiques modernes. L'intelligence artificielle (IA), couplée à la reconnaissance vocale et à l'automatisation, ouvre la voie à une nouvelle génération d'assistants intelligents capables d'agir et de réagir en fonction des besoins des utilisateurs, en rendant les environnements numériques plus intuitifs, accessibles et interactifs.

Dans ce contexte technologique en constante évolution, notre projet de fin d'études s'inscrit dans la dynamique des systèmes intelligents embarqués, en proposant une solution innovante : un **assistant robotisé intelligent pour desktop**, destiné à améliorer l'expérience utilisateur sur poste de travail. Le système, combinant logiciel embarqué et composantes physiques, permet à l'utilisateur d'interagir vocalement avec son environnement numérique, d'automatiser des tâches récurrentes, et de recevoir un retour visuel et physique à travers une interface graphique et des mouvements simulés.

Ce projet a été réalisé au sein de l'entreprise **Digi Growing**, spécialisée dans le développement de solutions technologiques innovantes, et ce, dans le cadre de l'obtention du diplôme de licence en Informatique et Multimédia à l'Institut Supérieur d'Informatique de Mahdia (ISIMa).

L'objectif principal du projet est de concevoir un robot assistant doté des fonctionnalités suivantes :

- Reconnaissance vocale locale, sans dépendance aux services cloud ;
- Interprétation intelligente des commandes grâce à des modules NLP embarqués ;
- Interaction physique à travers des servomoteurs pour simuler des réactions ;
- Interface utilisateur OLED pour afficher les informations en temps réel ;
- Architecture modulaire permettant une personnalisation avancée.

Pour répondre à ces objectifs, nous avons adopté une approche agile, basée sur la méthode **Scrum**, nous permettant de structurer notre travail en sprints successifs, assurant ainsi

un développement itératif, rapide et flexible.

Structure du mémoire

Ce mémoire est articulé en cinq chapitres. Il débute par le Chapitre 1 : **Contexte général du projet**, qui présente le cadre initial et les objectifs principaux du projet. Le Chapitre 2 : **Conception et Modélisation du Système** détaille la mise en place du système, en s'appuyant sur les diagrammes de classes, de séquence et de cas d'utilisation pour une modélisation précise. Le Chapitre 3 : **Réalisation du prototype** est consacré à la concrétisation du prototype, décrivant les étapes de sa fabrication et de son assemblage. Ensuite, le Chapitre 4 : **Modélisation IA** explique la conception de l'intelligence artificielle intégrée et les méthodes de gestion des données au sein du système. Enfin, le Chapitre 5 : **Design 3D Interface mobile** se focalise sur la conception en 3D du robot et le développement de son interface mobile, en mettant l'accent sur l'ergonomie et l'expérience utilisateur.

Enfin, une **conclusion générale** vient récapituler les acquis du projet et les perspectives d'évolution du système dans un contexte réel d'exploitation.

Chapitre 1

Contexte général du projet

Table des matières

1.1	Introduction	4
1.2	Présentation de l'entreprise	4
1.2.1	Digi Growing	4
1.2.2	Secteur d'activités	5
1.3	Cahier des charges	5
1.3.1	Besoins fonctionnels	5
1.3.2	Besoins non fonctionnels	5
1.3.3	Identification des acteurs	5
1.4	Méthode de développement	5
1.4.1	Méthode adoptée	6
1.4.2	Constitution de l'équipe	7
1.5	Étude de l'existant	7
1.6	Critique de l'existant	8
1.7	Présentation Générale	8
1.8	Objectifs de la solution	8
1.9	Architecture du système	9
1.10	Fonctionnement général	9
1.11	Avantages de la solution	9
1.12	Product Backlog	10
1.13	Organisation des Sprints	11
1.14	Détail des Sprints	11
1.14.1	Sprint 1 – Conception & Modélisation du système	11
1.14.2	Sprint 2 – Développement du système IoT	11
1.14.3	Sprint 3 – Modélisation IA et Gestion des données	12
1.14.4	Sprint 4 – Design 3D & Interface mobile	12
1.15	Critères d'acceptation des User Stories (Definition of Ready - DoR)	13
1.16	Conclusion	13

1.1 Introduction

L'interaction homme-machine est devenue un élément fondamental de l'expérience utilisateur dans les environnements numériques modernes. Cependant, malgré l'évolution des interfaces graphiques, l'utilisation d'ordinateurs reste souvent limitée à des interactions classiques clavier-souris. De nos jours, avec l'avancée rapide des technologies telles que l'Intelligence Artificielle (IA) et l'Internet des Objets (IoT) [17], il devient possible de concevoir des systèmes plus naturels et intuitifs.

Notre projet propose de tirer parti de ces technologies pour créer un **assistant robotisé intelligent pour ordinateur de bureau**. Ce système est capable de comprendre des commandes vocales, de répondre à l'utilisateur à travers une interface visuelle, et de simuler des réactions physiques à l'aide de composants robotiques. Il vise à enrichir et simplifier l'utilisation quotidienne de l'ordinateur, tout en offrant une solution modulaire, locale et personnalisable.

Dans ce chapitre, nous présentons l'entreprise d'accueil, le cadre général de notre projet, les objectifs visés par notre solution intelligente BEEMO, ainsi qu'un aperçu des solutions existantes. Nous expliquerons également les spécificités et avantages de notre approche face aux méthodes classiques.

1.2 Présentation de l'entreprise

1.2.1 Digi Growing

Digi Growing est une startup tunisienne fondée en 2024, spécialisée dans le développement de solutions innovantes dans les domaines du numérique, de l'IoT, et de l'intelligence artificielle. L'entreprise mise sur la créativité et l'intégration technologique pour accompagner ses clients dans la transformation digitale (voir Figure 1.1).



FIGURE 1.1 – Logo de Digi Growing

1.2.2 Secteur d'activités

Digi Growing intervient principalement dans :

Le développement de solutions embarquées intelligentes ; La conception d'interfaces homme-machine innovantes ; L'intégration de technologies IA dans des environnements réels ; Le prototypage rapide de systèmes robotiques et d'assistants virtuels.

1.3 Cahier des charges

1.3.1 Besoins fonctionnels

Le système proposé vise à répondre aux besoins suivants :

Comprendre et traiter des commandes vocales données par l'utilisateur ; Réagir avec des réponses visuelles sur un écran OLED intégré ; Simuler des expressions physiques (mouvement de la tête ou du corps) grâce à des servomoteurs ; Exécuter des actions locales comme l'ouverture de programmes, la lecture d'informations ou l'émission d'alertes ; Proposer une interaction fluide via une interface simple et intuitive.

1.3.2 Besoins non fonctionnels

Fonctionnement local sans dépendance à Internet (confidentialité assurée) ; Réactivité rapide aux commandes (latence faible) ; Système modulaire et évolutif pour intégrer de nouveaux modules (ex : agenda, météo, plugins) ; Affichage clair sur écran OLED et retour sonore agréable ; Faible consommation d'énergie pour les composants embarqués.

1.3.3 Identification des acteurs

Utilisateur final : Personne interagissant avec l'assistant via la voix ou l'interface ; **Assistant IA** : Module logiciel qui traite les requêtes, exécute les actions et fournit les réponses ; **Système embarqué** : Ensemble Raspberry Pi [13] + composants matériels (capteurs, servos, écran) ; **Développeur / Administrateur** : Responsable de la configuration, de la mise à jour et de l'enrichissement du système.

1.4 Méthode de développement

Pendant longtemps, les projets informatiques étaient gérés selon des méthodes classiques, basées sur une planification rigide et un découpage séquentiel des tâches. Cepen-

dant, avec l'évolution rapide du marché technologique et l'exigence croissante des clients, ces méthodes ont montré leurs limites.

C'est ainsi qu'à la fin des années 1990, les **méthodes agiles** ont émergé comme une alternative plus flexible et interactive. Elles privilégient la collaboration, l'adaptabilité, et l'implication continue du client tout au long du projet. Bien que conçues initialement pour l'informatique, ces méthodes sont désormais adoptées dans de nombreux domaines industriels.

1.4.1 Méthode adoptée

Afin d'obtenir un projet bien structuré et bien organisé, nous avons opté pour l'une des méthodes agiles : la méthode **SCRUM** [15]. Cette approche permet un pilotage du travail plus souple et sécurisé, ce qui facilite la conception et le développement de manière itérative et incrémentale (voir Figure 1.2).

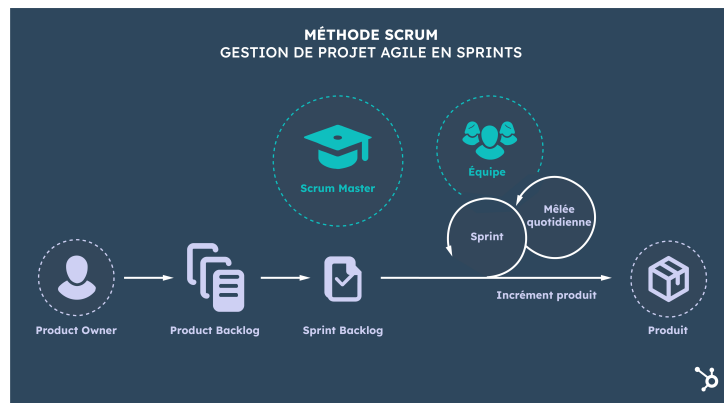


FIGURE 1.2 – Cycle Scrum basé sur des sprints successifs

Qu'est-ce que la méthode SCRUM ?

Parmi les méthodes agiles, la plus répandue est la méthode Scrum, fondée par *Ken Schwaber* et *Mike Beedle* en 2002 [15]. Elle vise à accroître la productivité des équipes en divisant le projet en cycles appelés **sprints**, chacun durant de quelques jours à un mois. Chaque sprint se termine par une démonstration des résultats obtenus et une rétrospective afin de corriger les erreurs et améliorer le processus de développement (voir Figure 1.3).

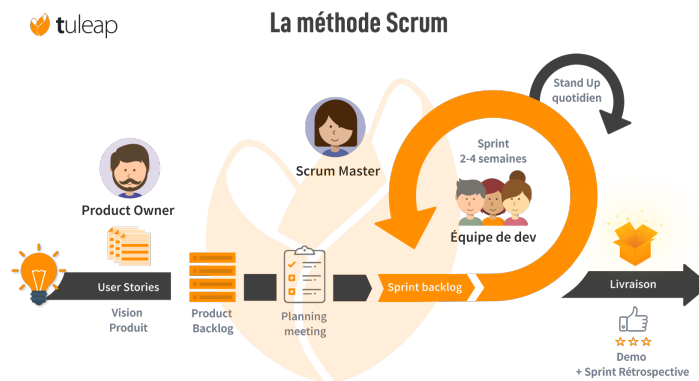


FIGURE 1.3 – Organisation d'un sprint Scrum

1.4.2 Constitution de l'équipe

Le cadre Scrum prévoit une répartition claire des rôles au sein de l'équipe :

Product Owner : l'encadrant professionnel, responsable de définir les besoins fonctionnels du projet. **Scrum Master** : l'encadrant technique, garant du bon déroulement de la méthode Scrum. **Développeurs** : l'équipe projet (le binôme), chargée de la réalisation concrète des tâches.

Les différents rôles au sein d'une équipe Scrum sont présentés en Figure 1.4.

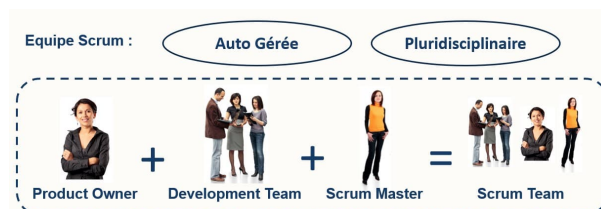


FIGURE 1.4 – Rôles dans une équipe Scrum

1.5 Étude de l'existant

L'étude de l'existant permet d'explorer les solutions actuelles, qu'elles soient commerciales ou open-source, dans le domaine des assistants vocaux et robotiques personnels. Elle permet de mieux comprendre leurs avantages, leurs limites, et les opportunités d'innovation.

Parmi les exemples les plus connus, on retrouve :

Siri (Apple), **Google Assistant** et **Cortana (Microsoft)** : intégrés dans les systèmes mobiles ou de bureau, ces assistants sont capable de répondre à des questions, d'ouvrir des applications ou de programmer des rappels. Ils nécessitent une connexion Internet constante et fonctionnent dans des environnements fermés.

Amazon Alexa : utilisé majoritairement dans la domotique, Alexa offre des interactions vocales mais dépend également du cloud. **Mycroft AI** : assistant vocal open-source personnalisable, mais sans composante physique ni retour visuel embarqué. **Jibo**, **Cozmo**, **Vector** : robots domestiques dotés de capacités expressives, mais souvent coûteux, propriétaires, ou orientés vers le divertissement.

Ces solutions présentent souvent des limitations :

Dépendance aux serveurs distants ; Accès restreint à la personnalisation ou au code source ; Absence d'interaction physique ou gestuelle ; Risques liés à la confidentialité des données.

1.6 Critique de l'existant

Cette analyse révèle un manque important dans le domaine des assistants de bureau intelligents qui soient :

Autonomes (fonctionnant localement sans cloud) ; Interactifs (voix, écran, mouvement) ; Accessibles et personnalisables (open-source, modulaire) ; Adaptés aux postes de travail standards.

C'est à partir de ces constats que nous avons défini les orientations de notre solution, en combinant intelligence logicielle, interaction physique, et exécution locale.

1.7 Présentation Générale

Afin de répondre aux limitations identifiées dans l'étude de l'existant, nous avons proposé de concevoir un **assistant robotisé intelligent pour ordinateur de bureau**, nommé **BEEMO**. Ce système offre une interaction homme-machine innovante en combinant traitement vocal, retour visuel et mouvements physiques.

Contrairement aux assistants classiques, BEEMO fonctionne en local, sans dépendance à Internet, ce qui garantit la confidentialité des échanges. Il est conçu pour être évolutif, modulaire, et facile à personnaliser selon les besoins de l'utilisateur.

1.8 Objectifs de la solution

La solution vise à :

Offrir une interaction vocale naturelle avec l'ordinateur ; Réaliser des actions sur commande (ouvrir des applications, afficher l'heure, annoncer la météo, etc.) ;

Proposer un retour visuel expressif via un écran OLED intégré ; Simuler des mouvements simples à l'aide de servomoteurs ; Fonctionner localement pour préserver la sécurité des données ; Être facilement extensible par ajout de nouveaux modules.

1.9 Architecture du système

Le système BEEMO est structuré autour de plusieurs composants :

Un Raspberry Pi 4 : plateforme centrale de traitement, hébergeant le code principal. **Un module de reconnaissance vocale** : basé sur *SpeechRecognition* et *vosk* [1], pour interpréter les commandes. **Un moteur logique intelligent** : capable de déclencher des actions selon les intentions détectées. **Un écran OLED** : utilisé pour afficher des messages, des animations ou des données système. **Des servomoteurs** : pour simuler des réactions physiques comme tourner la tête ou faire des gestes. **Une interface utilisateur** : conçue avec Tkinter pour faciliter le contrôle manuel et l'observation.

1.10 Fonctionnement général

Lorsqu'une commande vocale est détectée, le système :

Convertit la voix en texte ; Interprète l'intention via des règles locales ; Déclenche une action correspondante (exécution d'un script, affichage d'une réponse, mouvement, etc.) ; Fournit un retour visuel ou physique.

Ce processus assure une réponse rapide, intuitive et expressive, adaptée à un usage bureautique.

1.11 Avantages de la solution

Confidentialité : traitement local, aucune donnée envoyée à des serveurs externes ; **Accessibilité** : matériel peu coûteux, logiciels open-source ; **Personnalisation** : possibilité d'ajouter facilement des fonctionnalités selon les besoins ; **Interaction augmentée** : grâce à la combinaison voix, écran et mouvements.

Cette solution se positionne donc comme une alternative intelligente et locale aux assistants traditionnels, tout en ouvrant la voie à de futures améliorations dans les environnements de travail interactifs.

1.12 Product Backlog

Voici le backlog du produit proposé pour notre système d’assistant robotisé intelligent connecté. Il est structuré selon la méthode Scrum, sous forme de *user stories* priorisées (voir Tableau 1.1).

ID	User Story	Description	Priorité
PB1	Reconnaissance vocale locale	En tant qu'utilisateur, je veux que le robot reconnaisse mes commandes vocales localement pour préserver la confidentialité.	Très haute
PB2	Interaction via écran OLED	En tant qu'utilisateur, je veux que le robot affiche des réponses et des informations en temps réel sur un écran.	Haute
PB3	Communication avec l'application mobile	En tant qu'utilisateur, je veux pouvoir envoyer des commandes et recevoir des états depuis une application mobile.	Très haute
PB4	Détection d'événements via capteurs	En tant qu'utilisateur, je veux que le robot détecte les mouvements/températures/luminosité autour de lui pour réagir intelligemment.	Haute
PB5	Mouvements physiques (servos)	En tant qu'utilisateur, je veux que le robot puisse bouger la tête ou les bras pour rendre les interactions plus naturelles.	Moyenne
PB6	Connexion à une API IA (DeepSeek ou autre)	En tant qu'utilisateur, je veux que le robot comprenne mes requêtes complexes grâce à une IA connectée (DeepSeek, GPT, etc.) [4, 12].	Haute
PB7	Tableau de bord sur app mobile	En tant qu'admin, je veux surveiller l'état du robot (température, statut, logs) via une interface mobile.	Haute
PB8	Automatisation des réactions selon contexte	En tant qu'utilisateur, je veux que le robot réagisse automatiquement à des événements selon des règles IA définies.	Très haute

TABLE 1.1 – Backlog produit – Assistant IA robotisé connecté (User Stories)

1.13 Organisation des Sprints

Le projet est organisé selon **quatre** sprints principaux, chacun d’une durée de 30 jours, suivant la méthode Scrum [15]. Chaque sprint est orienté vers une livraison incrémentale des fonctionnalités (voir Tableau 1.2).

Sprint	Durée	Nom du Sprint
1	30 jours	Conception et Modélisation du système
2	30 jours	Développement du système IoT
3	30 jours	Modélisation IA et Gestion des données
4	30 jours	Design 3D et Interface mobile

TABLE 1.2 – Organisation des sprints du projet d’assistant connecté

1.14 Détail des Sprints

1.14.1 Sprint 1 – Conception & Modélisation du système

Diagrammes UML [11, 3] :

- Diagramme de cas d’utilisation (interaction utilisateur système IoT + app mobile)
 - Diagramme de classes (composants logiciels : modules IA, interface mobile, capteurs, etc.)
 - Diagramme de séquence (flux entre commande vocale → action sur le robot ou retour via app)
- **Objectif** : poser l’architecture du projet, structurer les composants logiciels et leurs interactions.

1.14.2 Sprint 2 – Développement du système IoT

Partie matérielle :

- Description des composants : RasPi, servomoteurs, capteurs environnementaux (température, humidité, gaz), écran OLED, module micro (pour Vosk)

– **Partie logicielle** :

Intégration de Vosk pour la reconnaissance vocale en local Développement des scripts embarqués pour le contrôle des moteurs et l’affichage Mise en place des communications (API REST via Flask [5] ou MQTT [10])

-
- **Objectif** : permettre une interaction vocale basique avec le robot et retour sur application mobile.

1.14.3 Sprint 3 – Modélisation IA et Gestion des données

Intelligence Artificielle :

- Préparation du dataset (commandes vocales, intentions utilisateur)
- Intégration d'un modèle IA distant (ex : DeepSeek, GPT via API) [4, 12]
- Traitement de la compréhension du langage et exécution des commandes (ouvrir porte, lumière, musique...)

- **Gestion des données** :

Structuration des flux de données entre mobile serveur robot Sécurisation des échanges

- **Objectif** : permettre au robot de comprendre des commandes complexes et de réagir via l'IA.

1.14.4 Sprint 4 – Design 3D & Interface mobile

Conception 3D :

- Modélisation du boîtier avec Fusion 360 (composants bien intégrés, ventilation, fixation)

- **Développement mobile** :

Interface en Flutter [7] : dashboard en temps réel, contrôle manuel, affichage capteurs Design UX/UI + sécurisation de l'app

- **Objectif** : finaliser l'intégration matérielle et offrir une interface mobile fonctionnelle et ergonomique.
-

1.15 Critères d'acceptation des User Stories (Definition of Ready - DoR)

Une **User Story** est considérée comme "*prête*" (*Ready*) lorsqu'elle répond à un ensemble de critères garantissant sa bonne compréhension et sa faisabilité par l'équipe de développement. Ces critères constituent la **Definition of Ready** (DoR).

Elle est **bien comprise** par toute l'équipe, sans ambiguïté. Elle comporte des **critères d'acceptation clairs et vérifiables**, permettant de confirmer sa bonne réalisation. Elle est **estimée et priorisée** dans le backlog produit. Elle **apporte une valeur métier** réelle et justifiée. Elle **ne dépend pas d'une autre tâche bloquante**, et peut être commencée immédiatement.

Ces critères garantissent une meilleure gestion du backlog et une planification réaliste des sprints dans le cadre de la méthode Scrum.

1.16 Conclusion

Ce chapitre a permis de contextualiser notre projet en présentant l'entreprise Digi Growing, le cahier des charges détaillé avec les besoins fonctionnels et non fonctionnels, ainsi que l'identification des acteurs. Nous avons également justifié le choix de la méthode agile Scrum pour la gestion de ce projet, en détaillant ses principes, l'organisation en sprints et la constitution de l'équipe. L'étude critique des solutions existantes a mis en évidence le besoin d'un assistant de bureau intelligent, local et interactif, ce qui a directement mené à la proposition de notre solution BEEMO. Enfin, nous avons esquissé l'architecture globale du système, son fonctionnement et ses avantages, pour finir par une présentation structurée du Product Backlog (voir Tableau 1.1) et de l'organisation en sprints (voir Tableau 1.2). Ce premier chapitre établit ainsi les fondations essentielles à la compréhension et à la poursuite du projet, en définissant clairement le "quoi" et le "comment" de notre démarche.

Chapitre 2

Sprint 1 - Conception et Modélisation du Système

Table des matières

2.1	Introduction	15
2.2	Analyse et Spécification des Besoins	15
2.2.1	Spécification des Besoins	15
2.2.2	Analyse des Besoins	15
2.2.3	Identification des Cas d'Utilisation	16
2.3	Diagramme de Cas d'Utilisation	16
2.3.1	Diagramme de Cas d'Utilisation Global	17
2.3.2	Diagramme de Cas d'Utilisation Détaillé	17
2.4	Diagramme de Classes	19
2.4.1	Introduction	19
2.4.2	Diagramme Principal	19
2.5	Diagrammes de Séquence	22
2.5.1	Diagramme de séquence principal	22
2.6	Diagramme de cas d'utilisation de l'application mobile BEEMO	23
2.6.1	Diagramme de Séquence de l'Application Mobile	27
2.7	Conclusion	29

2.1 Introduction

Dans ce chapitre, nous détaillerons les différents éléments de conception, notamment :

- Les besoins fonctionnels et non fonctionnels
- Les acteurs du système
- Les diagrammes de cas d'utilisation
- Le diagramme de classes
- Les diagrammes de séquence
- Les diagrammes d'activité

2.2 Analyse et Spécification des Besoins

2.2.1 Spécification des Besoins

Besoins Fonctionnels

Connexion/Déconnexion : L'utilisateur peut connecter/déconnecter le robot via une application mobile Commandes vocales : Interaction vocale pour contrôler le poste de travail Langage naturel : Dialogue intelligent en langage naturel
Paramétrage : Personnalisation des paramètres (volume, langue, etc.)

Besoins Non Fonctionnels

Sécurité : Authentification requise Fiabilité : Taux de disponibilité >99%
Performance : Temps de réponse <2s Confidentialité : Protection des données utilisateur

2.2.2 Analyse des Besoins

Identification des Acteurs

Les principaux acteurs identifiés sont présentés dans le diagramme des acteurs du système (voir Figure 2.1).

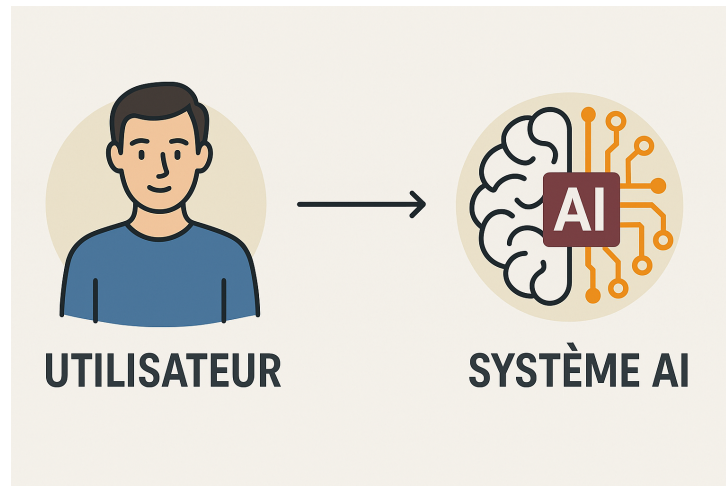


FIGURE 2.1 – Diagramme des acteurs du système

Utilisateur : L'utilisateur interagit avec le robot assistant via l'application mobile pour donner des commandes vocales et poser des questions. IA / Système : L'IA du système interprète les commandes et exécute les actions en fonction des instructions données par l'utilisateur.

2.2.3 Identification des Cas d'Utilisation

Les cas d'utilisation correspondent aux interactions entre les acteurs et le système. Le tableau 2.1 présente les principaux cas identifiés.

ID	Cas d'utilisation	Acteur(s) impliqué(s)
1	Connexion/Déconnexion du robot via une application mobile	Utilisateur
2	Commandes vocales	Utilisateur, IA
3	Interaction en langage naturel	Utilisateur, IA
4	Contrôle des paramètres	Utilisateur

TABLE 2.1 – Tableau descriptif "Identification des cas d'utilisations"

2.3 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation représente les fonctionnalités du système, les acteurs et leurs relations [3, 11]. Il permet de :

- Modéliser les besoins utilisateurs
 - Identifier les fonctionnalités principales
 - Délimiter les frontières du système
-

2.3.1 Diagramme de Cas d'Utilisation Global

Le diagramme de cas d'utilisation global du système est illustré dans la Figure 2.2.

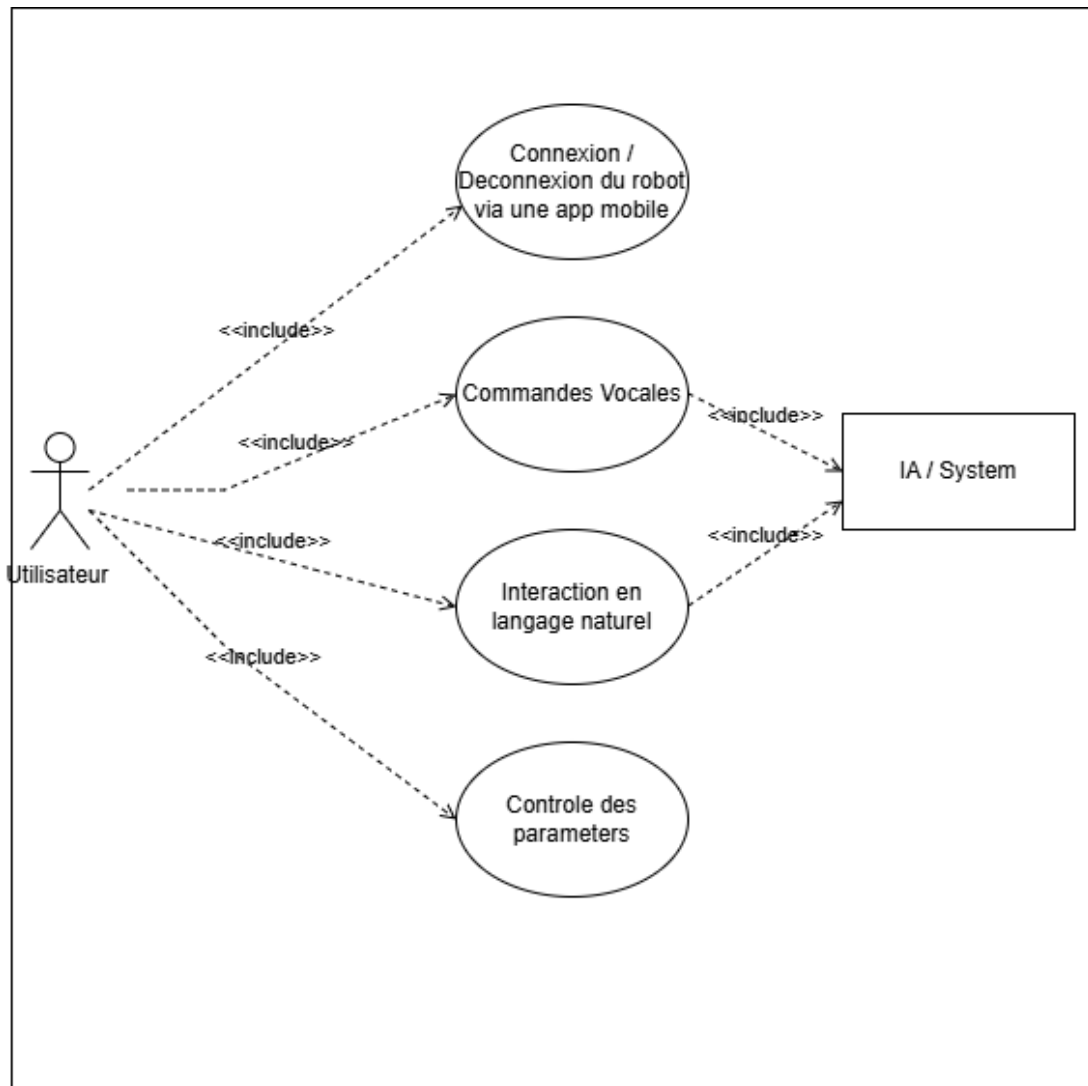


FIGURE 2.2 – Diagramme de cas d'utilisation global du système

Description des interactions :

Connexion/Déconnexion du robot via une application mobile	Commandes
vocales permettant à l'utilisateur de donner des ordres au robot	Interaction en
langage naturel pour un dialogue fluide et intuitif	Contrôle des paramètres pour
la personnalisation du système	

2.3.2 Diagramme de Cas d'Utilisation Détaillé

Le diagramme de cas d'utilisation détaillé du système est présenté dans la Figure 2.3.

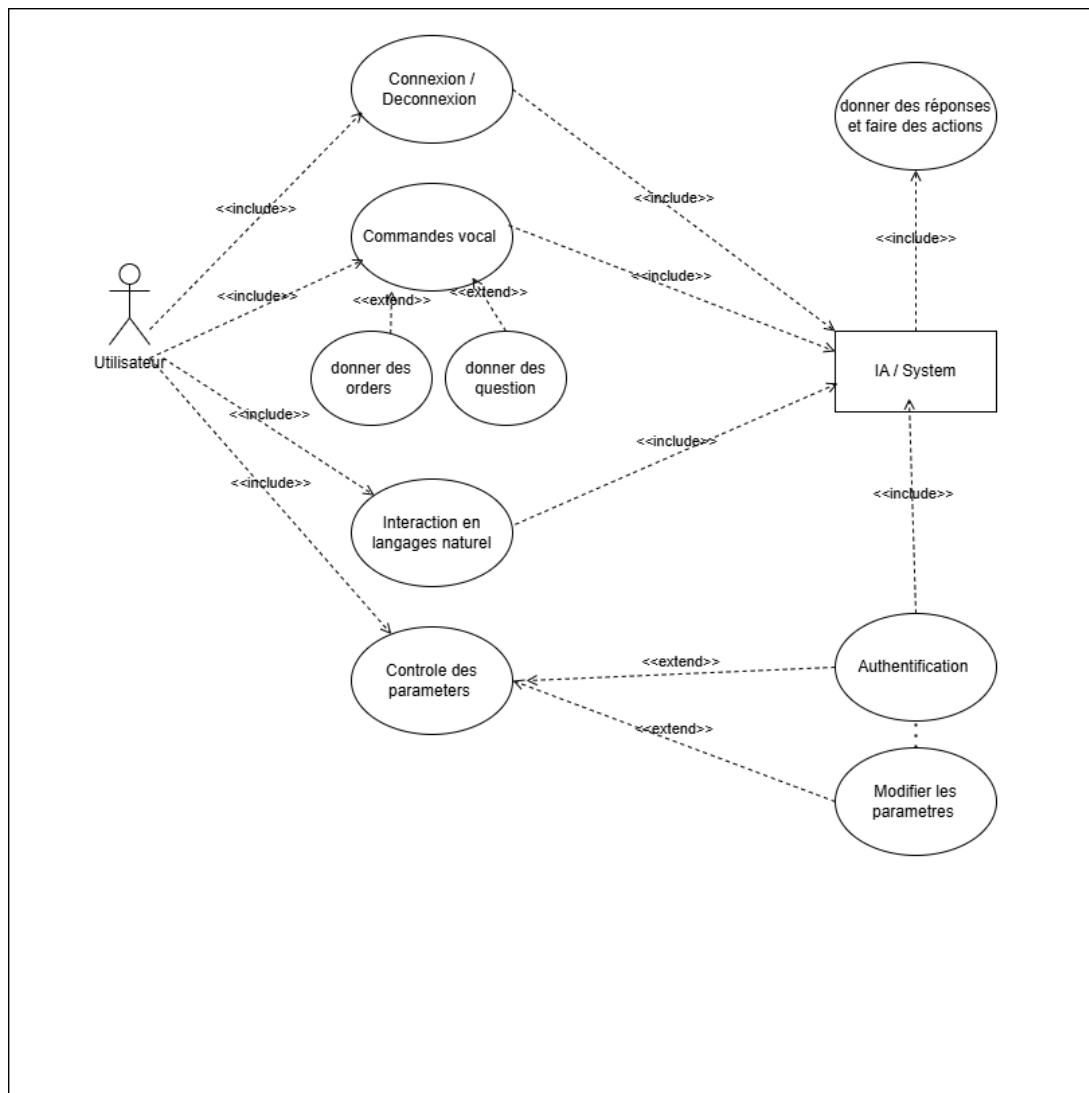


FIGURE 2.3 – Diagramme de cas d'utilisation détaillé du système

Acteurs Principaux :

Utilisateur : Initie toutes les interactions via commandes vocales ou l'interface mobile Système IA : Traite les requêtes et exécute les actions

Fonctionnalités Clés :

UC1 : Authentification

- Connexion sécurisée par token JWT
- Gestion des sessions utilisateur

— UC2 : Commandes Vocales

Traitement STT (Speech-to-Text) [1]
Exécution des actions système

Reconnaissance d'intention

— UC3 : Interaction Naturelle

Compréhension NLP avancée [12, 4]

Gestion du contexte dialogique

Dépendances :

«include» : L'authentification est requise pour accéder aux paramètres «extend» :
Les sous-commandes étendent les cas principaux

Flux Exceptionnels :

Reconnexion après expiration de session Gestion des commandes non reconnues
Réinitialisation des paramètres

2.4 Diagramme de Classes

2.4.1 Introduction

Le diagramme de classes fournit une vue statique de l'architecture du système, modélisant les principales entités et leurs relations [3, 11]. Cette représentation UML permet de :

Identifier les classes fondamentales du système Spécifier les attributs et opérations clés Définir les relations structurelles entre entités

2.4.2 Diagramme Principal

Le diagramme de classes du système d'assistant robotisé est illustré dans la Figure 2.4.

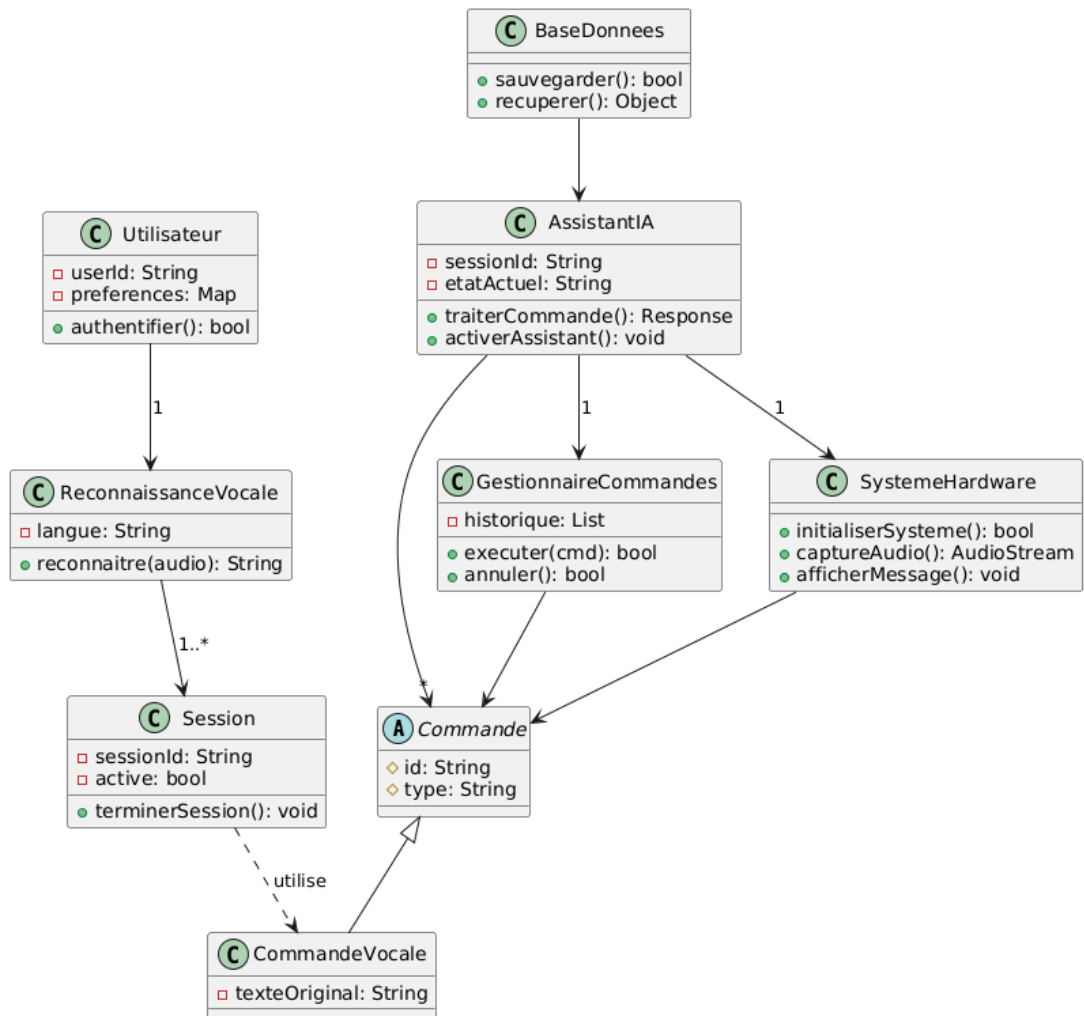


FIGURE 2.4 – Diagramme de classes du système d’assistant robotisé

Le diagramme ci-dessous présente l’architecture orientée objet du système BEEMO, un assistant vocal intelligent destiné à la domotique. Il modélise les principales entités du système, leurs attributs, leurs méthodes et les relations entre elles.

Utilisateur :

- *Attributs* : `userId: String`, `preferences: Map`
- *Méthode* : `authentifier(): bool`
- Représente l’utilisateur final du système, associé à la reconnaissance vocale.

— ReconnaissanceVocale :

Attribut : `langue: String` *Méthode* : `reconnaitre(audio): String`
Interface entre l’utilisateur et le système, convertit l’audio en texte.

— AssistantIA :

Attributs : sessionId: String, etatActuel: String *Méthodes* :
traiterCommande(): Response, activerAssistant(): void Cœur
décisionnel du système qui traite les commandes utilisateur.

— GestionnaireCommandes :

Attribut : historique: List *Méthodes* : executer(cmd): bool,
annuler(): bool Gère l'exécution et l'historique des commandes.

— Commande (abstraite) :

Attributs protégés : id: String, type: String Superclasse abstraite
des différentes commandes du système.

— CommandeVocale (hérite de Commande) :

Attribut : texteOriginal: String Représente une commande issue
de la reconnaissance vocale.

— Session :

- *Attributs* : sessionId: String, active: bool
- *Méthode* : terminerSession(): void
- Permet de suivre l'état d'une session utilisateur.

— SystèmeHardware :

Méthodes : initialiserSysteme(): bool, captureAudio(): AudioStream,
afficherMessage(): void Interface physique du système pour la capture
et l'affichage.

— BaseDonnees :

Méthodes : sauvegarder(): bool, recuperer(): Object Assure
la persistance des données.

En résumé, le diagramme met en lumière les interactions entre les composants logiciels et matériels du système, permettant à BEEMO d'interpréter et exécuter les commandes vocales d'un utilisateur dans un environnement domotique intelligent.

2.5 Diagrammes de Séquence

Le diagramme de séquence est une représentation UML qui décrit le comportement dynamique du système en modélisant les interactions entre les objets dans le cadre d'un cas d'utilisation spécifique [3, 11]. Cette section présente les principaux diagrammes de séquence associés à notre projet.

2.5.1 Diagramme de séquence principal

La Figure 2.5 illustre le flux d'interactions principal de notre système entre les différents acteurs et composants.

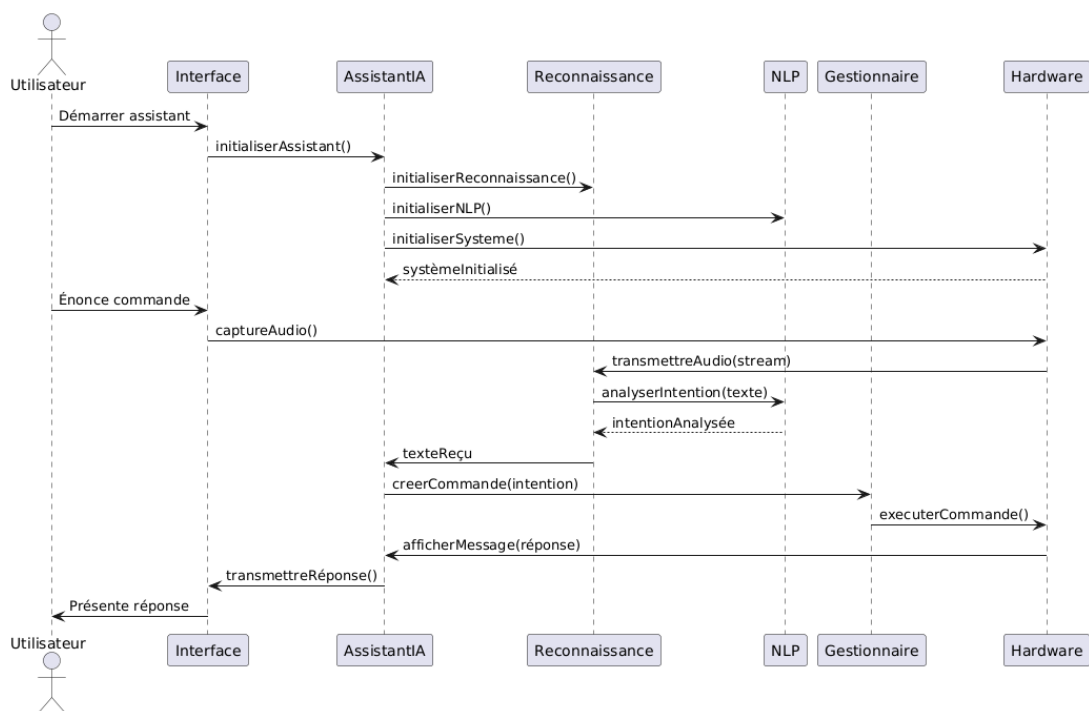


FIGURE 2.5 – Diagramme de séquence principal du système

Ce diagramme de séquence illustre le déroulement temporel d'une interaction typique entre l'utilisateur et le système BEEMO, un assistant vocal intelligent intégré à un environnement domotique. Il met en évidence l'enchaînement des messages échangés entre les composants logiciels et matériels pour traiter une commande vocale.

- **Utilisateur** : Démarre l'interaction avec le système en lançant l'assistant vocal.
 - **Interface** : Initie l'assistant via `initialiserAssistant()`.
 - **AssistantIA** : Une fois activé, il initialise les composants nécessaires :
 - `initialiserReconnaissance()` pour activer le module de reconnaissance vocale.
-

-
- `initialiserNLP()` pour activer le traitement du langage naturel.
 - **Hardware** : Est ensuite sollicité pour capturer l'audio via `captureAudio()`. Le flux audio est transmis au module de reconnaissance avec `transmettreAudio(stream)`.
 - **Reconnaissance** : Transforme l'audio en texte avec `texteReconnu` et le transmet au module NLP pour analyse sémantique via `analyserIntention(texte)`.
 - **NLP** : Extrait l'intention de la commande et retourne une structure analysée à l'Assistant via `intentionAnalysee`.
 - **AssistantIA** :
Formule une commande et la fait exécuter par le Gestionnaire.
 - **Hardware** : Affiche la réponse via `afficherMessage(reponse)`. Enfin, la réponse est retransmise à l'utilisateur via l'interface avec `transmettreReponse()`.

Ce scénario couvre une session utilisateur typique, depuis l'activation de l'assistant jusqu'à l'exécution de la commande et la présentation du résultat. Il montre clairement la coordination entre les sous-systèmes (interface, reconnaissance vocale, NLP, matériel, etc.) dans une architecture orientée services [5].

2.6 Diagramme de cas d'utilisation de l'application mobile BEEMO

Ce diagramme de cas d'utilisation modélise les interactions entre l'utilisateur et les fonctionnalités offertes par l'application BEEMO pour la gestion du robot et de l'environnement domestique intelligent, ainsi que par l'assistant de la plateforme PFE. Il a pour objectif de fournir une vue d'ensemble des capacités du système du point de vue de l'utilisateur (voir Figure 2.6).

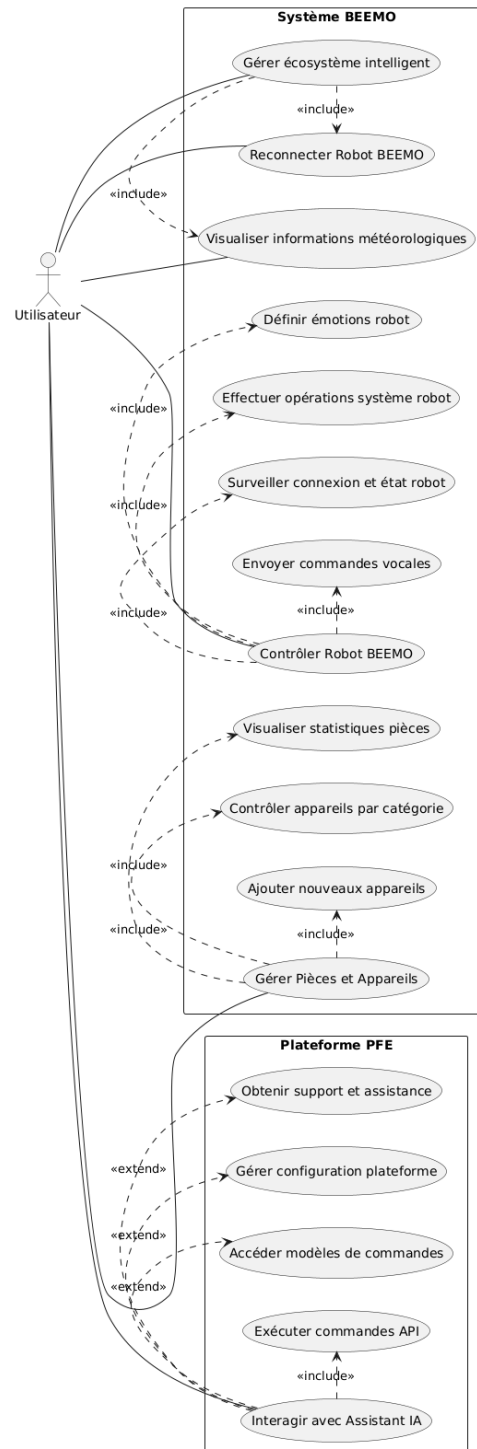


FIGURE 2.6 – Diagramme de cas d'utilisation de l'application mobile BEEMO

1. Acteur Principal

Utilisateur : Représente l'acteur externe principal qui interagit avec les deux systèmes. L'utilisateur initie toutes les actions et requêtes, qu'il s'agisse de contrôler le robot, de gérer les appareils domestiques, ou de communiquer avec l'assistant IA.

2. Systèmes Modélisés

3. Cas d'Utilisation et Leurs Rôles

Dans le Système BEEMO :

Gérer écosystème intelligent :

- «include» Reconnecter Robot BEEMO
- «include» Visualiser informations météorologiques

- Contrôler Robot BEEMO :

Dans la Plateforme PFE :

4. Relations

Le diagramme de classes ci-dessous représente l'architecture logicielle de l'application mobile BEEMO, modélisant les différentes entités, leurs attributs, leurs méthodes et les relations qu'elles entretiennent entre elles. Cette modélisation orientée objet constitue le socle architectural de notre système (voir Figure 2.7).

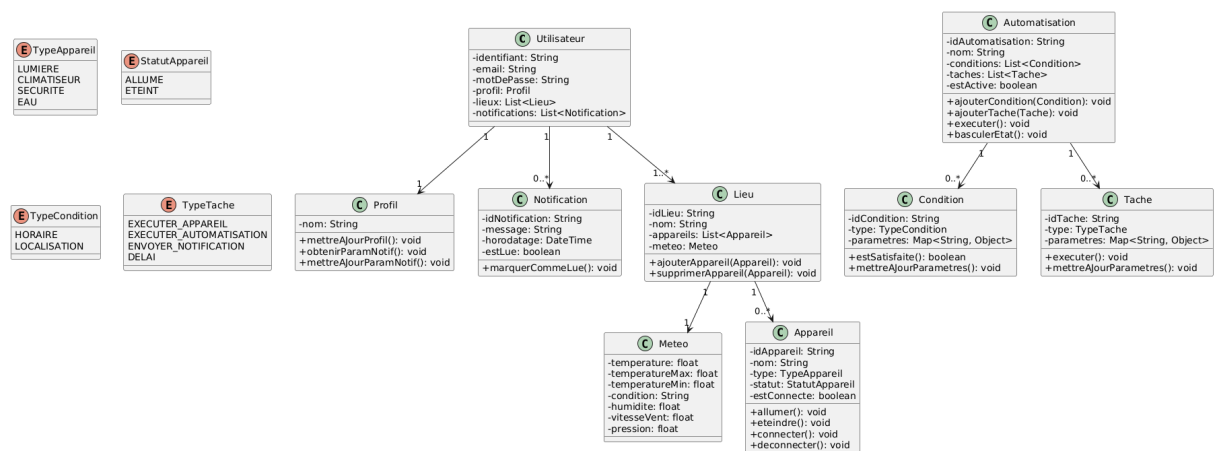


FIGURE 2.7 – Diagramme de classes de l'application mobile BEEMO

1. User :

- Responsabilité : Représente l'utilisateur de l'application et centralise ses informations personnelles
 - Attributs : userId (identifiant unique), email (adresse électronique), password (mot de passe crypté), profile (profil utilisateur), places (lieux gérés), notifications (alertes système)
-

-
- Relations : Agrège un profil personnel (Profile), plusieurs lieux (Places) et diverses notifications (Notifications)

2. Place :

- Responsabilité : Modélise un espace physique contrôlé par le système (pièce, zone, bâtiment)
- Attributs : placeId (identifiant unique), name (nom descriptif), devices (appareils associés), weatherInfo (données météorologiques locales)
- Méthodes : addDevice() (ajout d'appareil à l'espace), removeDevice() (retrait d'appareil)
- Relations : Contient plusieurs dispositifs (Device) et des informations météorologiques spécifiques (WeatherInfo)

3. Device :

- Responsabilité : Abstraction d'un appareil physique connecté au système
- Attributs : deviceId (identifiant unique), name (nom personnalisé), type (catégorie d'appareil), status (état actuel), isConnected (statut de connexion)
- Méthodes : turnOn() (activation), turnOff() (désactivation), connect() (établissement connexion), disconnect() (rupture connexion)
- Comportement : Répond aux commandes de l'utilisateur et signale son état au système

4. Automation :

- Responsabilité : Encapsule une règle d'automatisation programmée
- Attributs : automationId (identifiant unique), name (nom descriptif), conditions (déclencheurs), tasks (actions à exécuter), isActive (état d'activation)
- Méthodes : addCondition() (ajout condition), addTask() (ajout tâche), execute() (exécution), toggleActive() (activation/désactivation)
- Relations : Agrège plusieurs conditions (Condition) et tâches (Task) à exécuter
- Comportement : S'active automatiquement lorsque les conditions sont remplies

Compositions et Agrégations :

- User 1-1 Profile : Chaque utilisateur possède exactement un profil personnel (composition forte)
 - User 1-* Place : Un utilisateur peut gérer plusieurs lieux distincts (composition)
 - User 1-* Notification : L'utilisateur reçoit et conserve plusieurs notifications (composition)
 - Place 1-* Device : Un lieu contient plusieurs appareils qui peuvent exister indépendamment (agrégation)
-

-
- Place 1–1 WeatherInfo : Chaque lieu dispose d'informations météorologiques spécifiques (composition)
 - Automation 1–* Condition : Une automatisation comporte plusieurs conditions de déclenchement (composition)
 - Automation 1–* Task : Une automatisation comprend plusieurs tâches à exécuter (composition)

Types Énumérés :

- DeviceType : Catégorisation des appareils (LIGHT, AC, SECURITY, WATER_HEATER)
- DeviceStatus : États possibles des appareils (ON, OFF)
- ConditionType : Types de conditions d'automatisation (SCHEDULE, LOCATION)
- TaskType : Types d'actions automatisées (RUN_DEVICE, RUN_AUTOMATION, SEND_NOTIFICATION, DELAY_ACTION)

Patrons de conception utilisés :

- Observer : Utilisé pour la notification des changements d'état entre les composants
- Command : Implémenté dans la structure des Tasks pour encapsuler les actions
- Factory : Emploi de fabriques pour la création standardisée d'objets complexes
- Repository : Gestion de la persistance des données et de leur récupération
- Strategy : Utilisé pour interchanger les algorithmes de traitement des conditions

2.6.1 Diagramme de Séquence de l'Application Mobile

Le diagramme de séquence ci-dessous modélise les interactions temporelles entre l'utilisateur et les différents services du système BEEMO, illustrant le flux dynamique des opérations dans plusieurs scénarios d'utilisation clés (voir Figure 2.8).

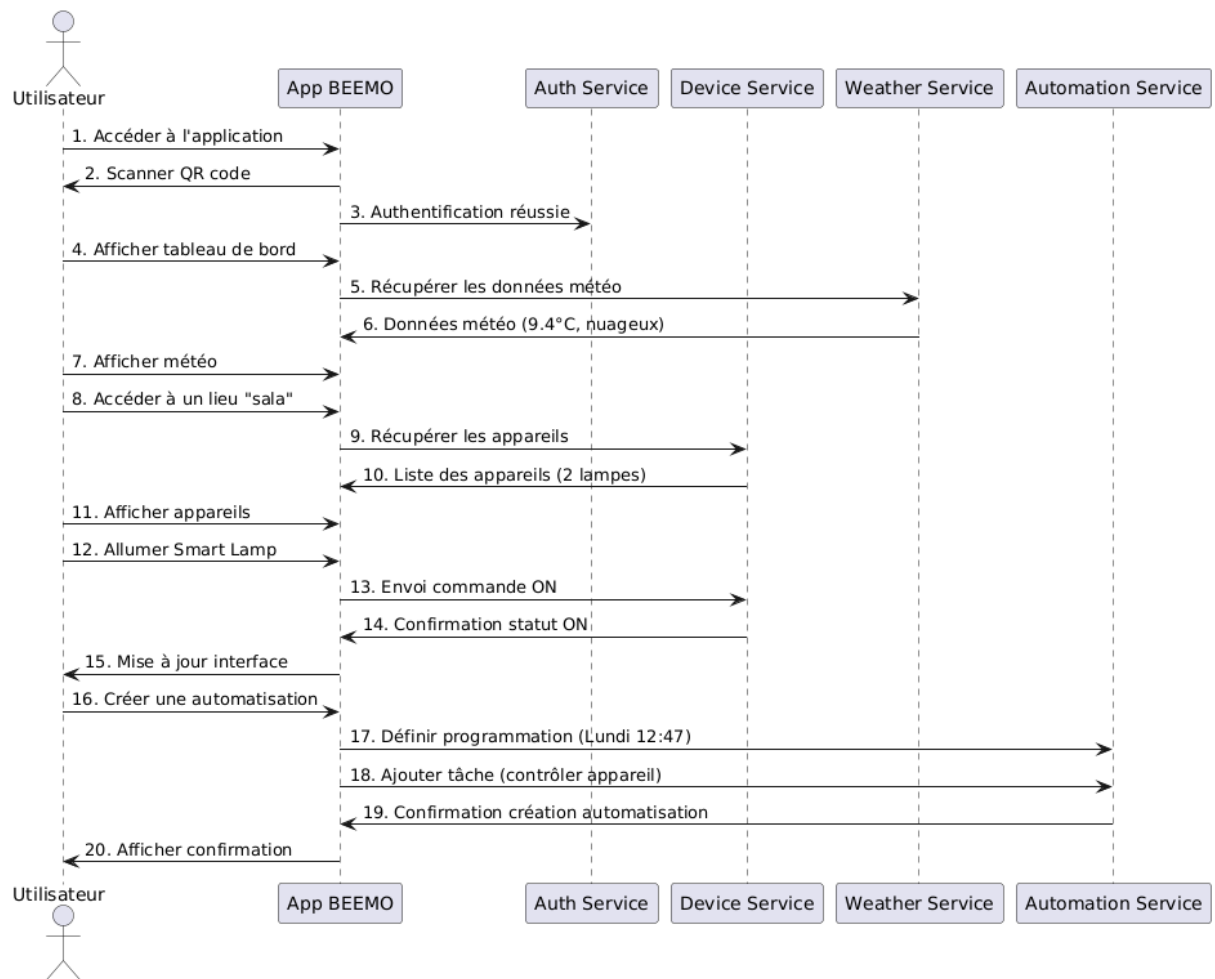


FIGURE 2.8 – Diagramme de séquence de l'application mobile BEEMO

1. Processus d'Authentification :
2. Récupération des Données Météorologiques :
3. Gestion et Contrôle des Appareils :
4. Configuration d'Automatisation :

L'application BEEMO utilise une architecture orientée services (SOA) qui offre plusieurs avantages majeurs [5] :

- Modularité : Chaque service (Auth, Device, Weather, Automation) fonctionne de manière indépendante
- Évolutivité : Possibilité d'ajouter ou de modifier des services sans impacter l'ensemble du système
- Robustesse : Un dysfonctionnement dans un service n'affecte pas l'intégralité de l'application

-
- Réutilisabilité : Les services peuvent être exploités par d'autres interfaces (application web, API)
 - Maintenance simplifiée : Les équipes peuvent travailler sur différents services en parallèle

Cette architecture facilite l'intégration avec le robot assistant tout en garantissant une expérience utilisateur fluide et réactive.

2.7 Conclusion

Au terme de ce premier sprint, nous avons posé les bases architecturales et fonctionnelles de notre assistant robotisé intelligent pour desktop. L'analyse détaillée des besoins a permis d'identifier clairement les fonctionnalités clés, telles que la reconnaissance vocale, l'interaction en langage naturel et le contrôle des paramètres. La modélisation UML, à travers les diagrammes de cas d'utilisation (voir Figure 2.2 et Figure 2.3), de classes (voir Figure 2.4 et Figure 2.7) et de séquence (voir Figure 2.5 et Figure 2.8), a fourni une vue structurée des composants du système (y compris l'application mobile BEEMO) et de leurs interactions. Ces étapes de conception sont cruciales pour guider la phase de réalisation future et garantir que le développement s'aligne avec les exigences définies. Ce sprint a ainsi jeté les fondations solides nécessaires à la construction progressive de notre solution.

Chapitre 3

Sprint - 2 : Réalisation du prototype

Table des matières

3.1	Introduction	31
3.2	Environnement logiciel	31
3.3	Environnement matériel	33
3.4	Installation et configuration des cartes	37
3.5	Architecture générale du système	39
3.6	Schéma de câblage du prototype	41
3.7	Prototypes du robot BEEMO	43
3.8	Conclusion	44

3.1 Introduction

Cette phase représente la concrétisation du projet. Dans ce chapitre, nous présentons l'environnement de développement utilisé (logiciel et matériel), ainsi que les différents composants techniques qui ont permis la mise en œuvre du système embarqué.

Voici les grandes lignes de ce chapitre :

Environnement logiciel : outils utilisés pour le développement, la modélisation, la conception d'interfaces et l'intelligence artificielle. **Environnement matériel** : choix des composants physiques pour construire et tester le prototype embarqué.

3.2 Environnement logiciel

L'environnement logiciel utilisé dans ce projet est composé de plusieurs outils essentiels pour la modélisation, le développement, l'interface utilisateur, le stockage, et le traitement des données.

UML (Unified Modeling Language) UML est un langage de modélisation visuel utilisé pour spécifier, visualiser, construire et documenter les composants d'un système logiciel [3, 11]. Il a permis de définir clairement les cas d'utilisation, les diagrammes de classes et les flux du système avant la phase de développement (voir Figure 3.1).



FIGURE 3.1 – Logo UML

Visual Studio Code (VS Code) Visual Studio Code est un éditeur de code source léger mais puissant, développé par Microsoft. Il prend en charge de nombreux langages de programmation et propose des extensions utiles pour le développement logiciel. Dans notre projet, il a été utilisé pour l'édition de code, l'organisation du projet et l'intégration avec Git (voir Figure 3.2).



FIGURE 3.2 – Logo Visual Studio Code

Flutter Flutter est un framework open-source développé par Google pour créer des applications mobiles multiplateformes [7]. Il a été utilisé pour concevoir l'interface utilisateur mobile du projet (voir Figure 3.3).



FIGURE 3.3 – Logo Flutter

Firebase Firebase est une plateforme de Google offrant des services backend comme l'authentification, la base de données en temps réel, et l'hébergement. Elle a été utilisée pour stocker et synchroniser les données de l'application mobile (voir Figure 3.4).



FIGURE 3.4 – Logo Firebase

LaTeX LaTeX est un langage de composition de documents utilisé pour produire des rapports professionnels. Il a été utilisé pour la rédaction et la mise en forme de ce rapport (voir Figure 3.5).



FIGURE 3.5 – Logo LaTeX

Figma Figma est un outil de design collaboratif en ligne. Il a été utilisé pour la conception des maquettes de l'interface utilisateur du projet mobile (voir Figure 3.6).



FIGURE 3.6 – Logo Figma

Kaggle Kaggle est une plateforme spécialisée en data science et apprentissage automatique. Elle a été utilisée pour entraîner et tester les modèles d'intelligence artificielle du projet (voir Figure 3.7).



FIGURE 3.7 – Logo Kaggle

3.3 Environnement matériel

Pour un projet plus fonctionnel et réalisable, on doit choisir les bons matériels adaptés à nos besoins. Pour cette raison, nous avons commencé par le choix de la carte de développement, puis nous avons sélectionné les capteurs, actionneurs, interfaces et modules nécessaires à la concrétisation du système embarqué.

Raspberry Pi 4 (8 Go RAM) Le Raspberry Pi 4 est une carte de développement puissante, capable d'exécuter un système d'exploitation complet, idéale pour les applications embarquées nécessitant des ressources avancées comme l'interface graphique ou le traitement de données [13] (voir Figure 3.8).



FIGURE 3.8 – Carte Raspberry Pi 4

Écran LCD 2" (IPS, 240×320) Cet écran couleur est utilisé pour l’affichage d’informations en temps réel. Sa technologie IPS offre un bon angle de vision et une bonne lisibilité (voir Figure 3.9).

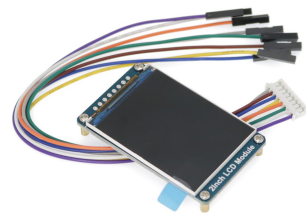


FIGURE 3.9 – Écran LCD 2 pouces IPS

Haut-parleurs 8 0.5W Deux petits haut-parleurs ont été utilisés pour permettre la sortie audio du système, notamment pour les signaux vocaux ou sonores (voir Figure 3.10).



FIGURE 3.10 – Haut-parleurs 8

Microphone MEMS INMP441 Ce microphone numérique est utilisé pour la capture audio. Il offre une grande sensibilité et une compatibilité directe avec les interfaces I2S du Raspberry Pi (voir Figure 3.11).

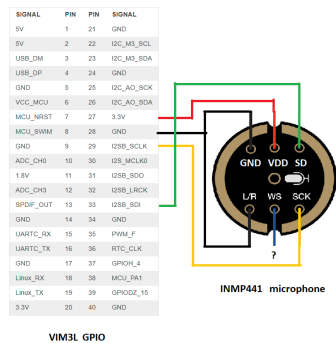


FIGURE 3.11 – Microphone MEMS INMP441

Amplificateur PAM8403 Ce module amplificateur stéréo (2x3W) permet d'alimenter les haut-parleurs avec un bon rendu sonore, tout en consommant peu d'énergie [8] (voir Figure 3.12).



FIGURE 3.12 – Amplificateur PAM8403

Capteur de vibration SW-420 Ce capteur détecte les vibrations physiques, utile pour détecter les chocs ou les mouvements mécaniques (voir Figure 3.13).

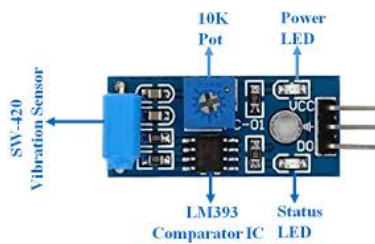


FIGURE 3.13 – Capteur de vibration SW-420

Module tactile capacitif TTP223B Ce module remplace un bouton mécanique par une interface tactile capacitive, ce qui améliore l'ergonomie du dispositif (voir Figure 3.14).

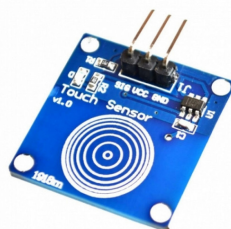


FIGURE 3.14 – Module tactile TTP223B

Carte driver servos (16 canaux, I2C) Cette carte permet de piloter jusqu'à 16 servomoteurs grâce à une communication I2C avec le Raspberry Pi. Elle facilite le contrôle de plusieurs actionneurs de manière simultanée (voir Figure 3.15).



FIGURE 3.15 – Carte driver de servomoteurs

Servomoteur MG90D (360°) Ce servomoteur est utilisé pour les mouvements nécessitant une rotation continue. Il offre une puissance suffisante pour de petites charges (voir Figure 3.16).



FIGURE 3.16 – Servomoteur MG90D

Mini servomoteur SG90 Ces petits servomoteurs sont utilisés pour les actions de rotation précises à 180°, comme les leviers ou les volets mécaniques (voir Figure 3.17).

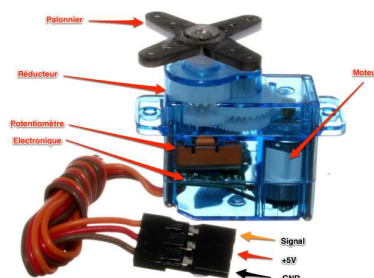


FIGURE 3.17 – Mini servomoteur SG90

Plaque perforée 7x9 Support pour assembler les composants de manière stable et sans soudure. Elle est utilisée pour le prototypage (voir Figure 3.18).



FIGURE 3.18 – Plaque perforée de prototypage

Convertisseur Micro USB to 5V Ce module permet d’alimenter l’ensemble du système à partir d’un port USB en fournissant une tension stable de 5V (voir Figure 3.19).



FIGURE 3.19 – Convertisseur Micro USB vers 5V

3.4 Installation et configuration des cartes

Installation du système d’exploitation

Raspberry Pi Imager est une application conçue pour simplifier l’installation de systèmes d’exploitation compatibles avec les cartes Raspberry Pi [13]. Elle offre une gamme d’options téléchargeables directement depuis le site officiel de la Fondation Raspberry, notamment Raspberry Pi OS (anciennement Raspbian). Elle permet également d’écrire des images personnalisées sur une carte microSD.

Pour installer le système d’exploitation avec Raspberry Pi Imager, il faut suivre les étapes suivantes :

Télécharger et installer **Raspberry Pi Imager** depuis le site officiel : <https://www.raspberrypi.com/software>. Lancer l’application, sélectionner un système d’exploitation (par exemple : Raspberry Pi OS Lite). Choisir la

carte SD comme périphérique de destination. Cliquer sur "Write" pour lancer l'installation de l'image sur la carte SD.

L'interface de Raspberry Pi Imager pour le choix de l'OS est présentée dans la Figure 3.20.

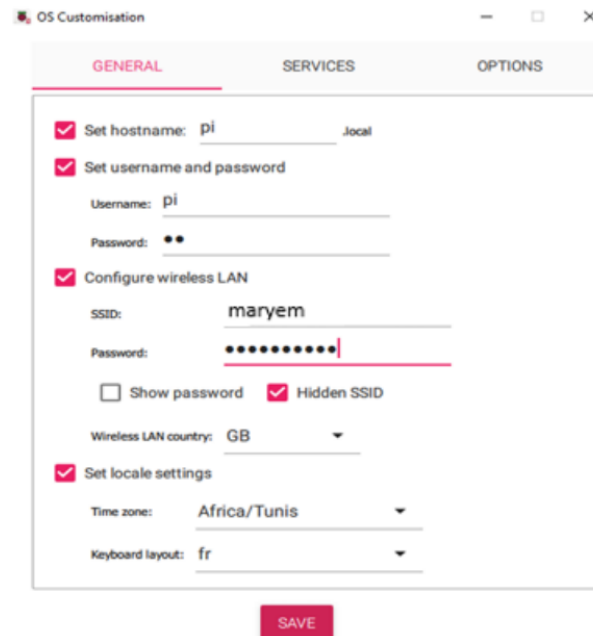


FIGURE 3.20 – Interface de Raspberry Pi Imager (choix de l'OS)

L'écriture de l'image sur la carte SD est visualisée dans la Figure 3.21.

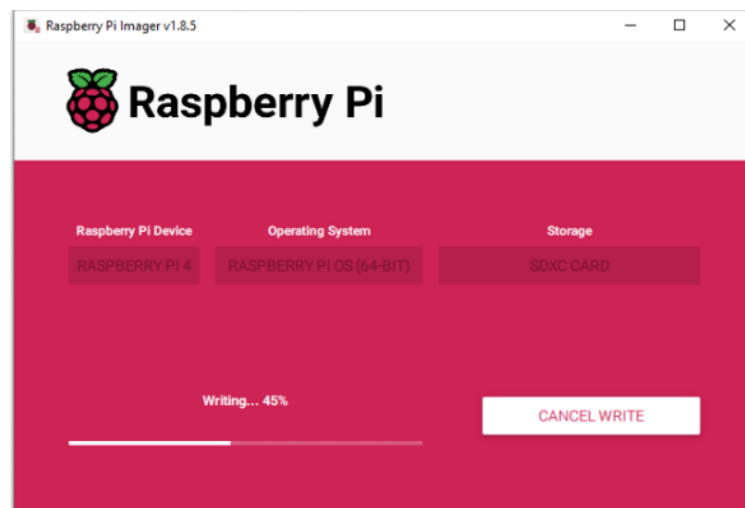


FIGURE 3.21 – Écriture de l'image sur la carte SD avec Raspberry Pi Imager

Une fois l'écriture de l'image terminée, on insère la carte microSD dans le port dédié du Raspberry Pi 4, puis on connecte l'alimentation pour démarrer l'appareil.

Connexion à distance avec VNC Viewer

Pour permettre la communication entre le Raspberry Pi et un PC, notamment l’affichage de l’écran sans moniteur HDMI, nous avons utilisé le logiciel **VNC Viewer**.

Ce logiciel permet d’accéder à l’interface graphique du Raspberry Pi via le réseau local.

Voici les étapes de configuration :

Activer l’option "VNC" dans **raspi-config** sur le Raspberry Pi. Installer VNC Viewer sur le PC (disponible sur <https://www.realvnc.com>). Se connecter en entrant l’adresse IP locale du Raspberry Pi et les identifiants par défaut.

La connexion à l’écran du Raspberry Pi via VNC Viewer est montrée dans la Figure 3.22.

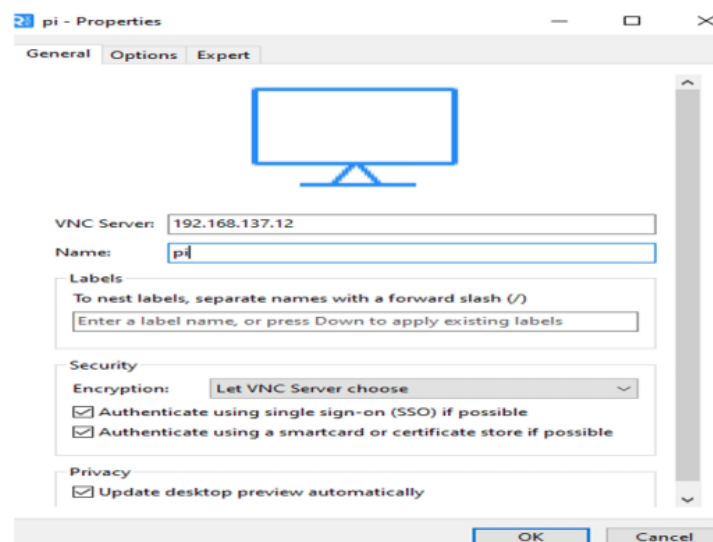
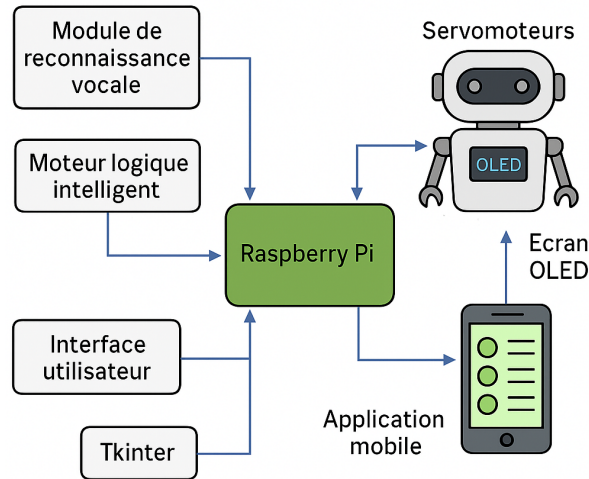


FIGURE 3.22 – Connexion à l’écran du Raspberry Pi via VNC Viewer

3.5 Architecture générale du système

L’architecture générale de notre système repose sur une organisation modulaire qui permet une interaction intelligente et fluide entre les composants matériels et logiciels. Le système BEEMO vise à offrir une expérience utilisateur intuitive à travers une interface vocale, un affichage visuel dynamique, et une application mobile connectée.

L’architecture générale du système BEEMO est présentée dans la Figure 3.23.



Architecture du système BEEMO

FIGURE 3.23 – Architecture générale du système BEEMO

L'architecture se compose des éléments suivants :

— **Raspberry Pi 4 :**

Sert de centre de contrôle du robot. Il coordonne les entrées/sorties des différents modules (voix, écran, servos, etc.) [13].

— **Servomoteurs :**

Contrôlent les mouvements des bras du robot pour simuler des gestes humains lors des interactions.

— **Écran OLED :**

Affiche un visage animé qui reflète les émotions ou réactions du robot, selon les stimuli reçus.

— **Capteur de vibration :**

Détecte un contact ou une présence physique. Lorsque le capteur est activé, BEEMO change d'expression faciale et émet un son via le haut-parleur.

— **Module de reconnaissance vocale :**

Permet à l'utilisateur d'interagir oralement avec le robot. Les commandes vocales sont traitées localement et analysées par le moteur logique [1].

— **Moteur logique intelligent :**

Analyse les intentions de l'utilisateur, traite les commandes, et détermine les réactions appropriées (affichage, mouvement, réponse vocale).

— **Interface utilisateur (Tkinter) :**

Permet un contrôle manuel du robot via un écran tactile ou un PC. Elle facilite le développement et le test du système.

- **Application Flutter :**

Connectée au robot via un identifiant unique, elle permet de visualiser les informations, les modules connectés et d'envoyer des instructions simples [7].

- **Haut-parleur :**

Utilisé pour répondre à l'utilisateur avec des sons ou des messages vocaux selon les situations.

Grâce à cette architecture, BEEMO combine intelligence embarquée, interaction vocale, interface visuelle expressive, et connectivité mobile pour créer une expérience utilisateur riche et engageante [17].

3.6 Schéma de câblage du prototype

Cette section illustre le câblage complet du robot BEEMO, reliant tous les composants matériels essentiels au fonctionnement du système : Raspberry Pi, écran OLED, servomoteurs, microphone, haut-parleur, capteur de vibration, et plus encore.

Le schéma de câblage du robot BEEMO est présenté dans la Figure 3.24.

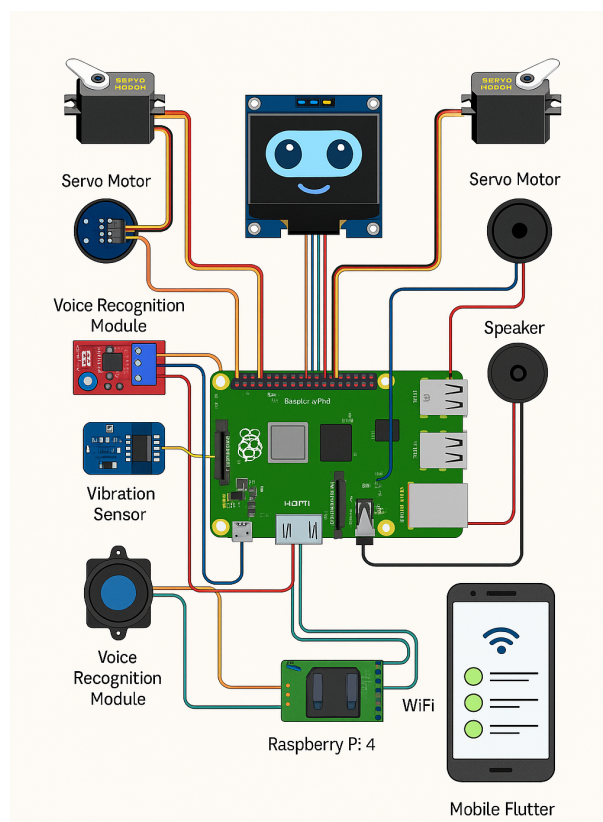


FIGURE 3.24 – Schéma de câblage du robot BEEMO

Le schéma ci-dessus montre les connexions suivantes :

- **Raspberry Pi 4** : centre de traitement, connecté aux composants via les broches GPIO.
- **Écran OLED** : connecté via bus I2C (SDA, SCL).
- **Servomoteurs** : connectés aux broches PWM pour contrôler les bras.
- **Microphone** : module audio utilisé pour la reconnaissance vocale.
- **Haut-parleur** : restitue les réactions vocales ou sons expressifs.
- **Capteur de vibration** : déclenche des animations et sons lorsqu'un choc est détecté.
- **Alimentation** : source 5V stable alimentant le Pi et les servomoteurs.

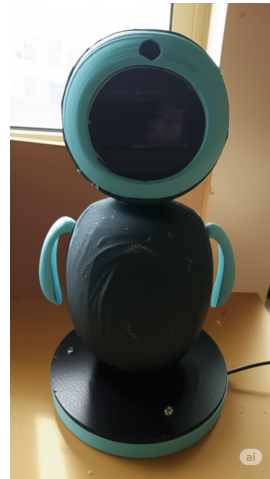
Chaque composant interagit avec le Raspberry Pi afin d'assurer une coordination fluide entre les actions physiques, l'affichage et les réponses vocales.

3.7 Prototypes du robot BEEMO

Les figures suivantes présentent différentes vues du prototype physique du robot BEEMO, mettant en évidence son design compact et les composants clés intégrés dans sa structure. Ce prototype sert de base pour les interactions utilisateur et les tests fonctionnels du système embarqué (voir Figure ??).



(a) Vue frontale du prototype du robot BEEMO, montrant l'écran intégré et les bras articulés.



(b) Vue légèrement inclinée du prototype BEEMO, mettant en évidence les finitions et la compacité.



(c) Vue de face rapprochée du prototype, avec un aperçu des éléments du visage du robot.



(d) Vue latérale du prototype du robot BEEMO, illustrant sa silhouette générale.

FIGURE 3.25 – Différentes vues du prototype physique du robot BEEMO.

3.8 Conclusion

Ce chapitre a détaillé la mise en œuvre concrète du système BEEMO, en couvrant à la fois les environnements logiciel et matériel nécessaires à son fonctionnement. Nous avons présenté les outils de développement utilisés pour la conception de l'application mobile, l'interface utilisateur et l'intelligence artificielle, ainsi que les composants électroniques sélectionnés pour construire un prototype fonctionnel, interactif et autonome. L'architecture globale du système (voir Figure 3.23) ainsi que le schéma de câblage (voir Figure 3.24) ont permis d'illustrer la manière dont chaque élément est intégré pour assurer une interaction fluide entre l'utilisateur et le robot. Grâce à cette phase de réalisation, nous avons pu transformer notre idée initiale en un prototype opérationnel (voir Figure ??), posant ainsi les bases pour les phases suivantes de test, d'optimisation et de validation du système.

Chapitre 4

Sprint - 3 : Modélisation IA

Table des matières

4.1	Intelligence Artificielle (IA)	46
4.1.1	Machine Learning	46
4.1.2	Deep Learning	46
4.1.3	Computer Vision	47
4.2	Réseaux de Neurones Convolutifs (CNN)	47
4.2.1	Couche Convulsive	48
4.2.2	Architecture du traitement vocal basé sur l'IA	48
4.2.3	Analyse des métriques d'entraînement	49
4.3	Conclusion	56

4.1 Intelligence Artificielle (IA)

L'intelligence artificielle (IA) désigne la capacité d'un système informatique à exécuter des tâches nécessitant habituellement l'intelligence humaine [14]. Elle se divise en plusieurs sous-domaines, chacun utilisant des méthodes spécifiques selon les objectifs visés. Parmi les domaines les plus pertinents pour notre projet, on retrouve le Machine Learning (apprentissage automatique) [2], le Deep Learning (apprentissage profond) [6], et la Computer Vision (vision par ordinateur) [16].

Les sous-domaines de l'intelligence artificielle sont présentés dans la Figure 4.1.

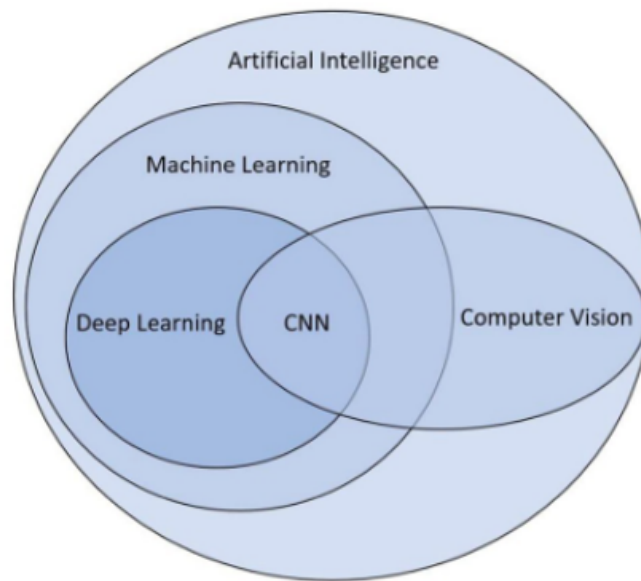


FIGURE 4.1 – Les sous-domaines de l'intelligence artificielle

4.1.1 Machine Learning

Le Machine Learning est une branche de l'IA qui vise à concevoir des systèmes capables d'apprendre à partir des données, puis d'ajuster automatiquement leur comportement [2]. Dans notre projet, il est utilisé pour modéliser et prédire des comportements récurrents, notamment dans les scénarios de détection d'objets ou de mouvement. Parmi les techniques explorées, nous retrouvons la régression linéaire, la régression logistique, l'algorithme K-Nearest Neighbors (KNN), ainsi que les machines à vecteurs de support (SVM).

4.1.2 Deep Learning

Le Deep Learning est une méthode d'apprentissage automatique reposant sur des réseaux de neurones profonds. Grâce à sa structure multi-couches, il permet de détecter

des motifs complexes dans de grands ensembles de données, notamment visuelles [6]. Les principales techniques utilisées dans notre projet comprennent les réseaux de neurones convolutionnels (CNN), les réseaux de neurones récurrents (RNN), et les réseaux générateurs adverses (GAN).

4.1.3 Computer Vision

La Computer Vision, ou vision par ordinateur [16], consiste à permettre aux machines d'analyser, d'interpréter et de comprendre des données visuelles. Elle repose sur des algorithmes capables d'extraire automatiquement des informations pertinentes à partir d'images ou de vidéos. Dans notre système, elle constitue le noyau de l'interaction homme-machine, notamment à travers la reconnaissance d'objets et l'analyse de scènes complexes.

4.2 Réseaux de Neurones Convolutifs (CNN)

Les Convolutional Neural Networks (CNN) [9] sont des modèles de Deep Learning particulièrement adaptés aux tâches de classification d'images. Depuis les années 1980, ils ont permis de grandes avancées dans la reconnaissance d'objets et de motifs visuels. Ils utilisent des concepts d'algèbre linéaire, notamment la multiplication de matrices, pour extraire automatiquement des caractéristiques locales à partir des pixels d'une image.

La structure d'un réseau de neurones convolutif est illustrée dans la Figure 4.2.

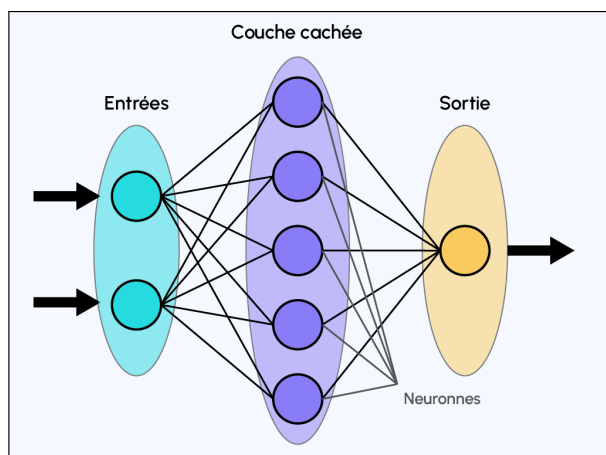


FIGURE 4.2 – Structure d'un réseau de neurones convolutif

En plus des applications en vision par ordinateur, les CNN sont aujourd'hui utilisés dans d'autres domaines comme la reconnaissance vocale ou le traitement du langage naturel, démontrant leur polyvalence et leur efficacité.

4.2.1 Couche Convolutive

La couche convolutive [9] est la composante principale d'un CNN. Elle applique des filtres (ou noyaux) sur l'entrée pour détecter des caractéristiques spécifiques comme les contours, textures ou formes géométriques. Ces filtres se déplacent sur toute la surface de l'image, une opération appelée convolution, afin d'extraire une carte de caractéristiques (feature map) qui met en évidence les éléments pertinents pour la classification.

4.2.2 Architecture du traitement vocal basé sur l'IA

La figure suivante présente l'architecture globale du pipeline de traitement vocal mis en œuvre dans notre système. Ce schéma illustre les différentes étapes nécessaires à la compréhension et à l'exécution d'une commande vocale, depuis la capture de l'audio jusqu'à la réponse intelligente générée par l'assistant. Chaque bloc représente un module clé intervenant dans ce processus complexe de traitement de la parole (voir Figure 4.3).

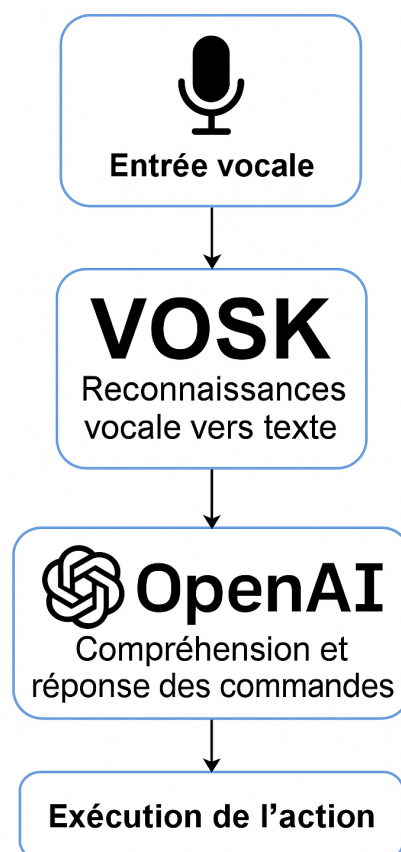


FIGURE 4.3 – Architecture de traitement de commande vocale

Cette architecture suit une démarche structurée en plusieurs étapes distinctes. Tout

d'abord, l'utilisateur interagit avec le système à l'aide d'une commande vocale en anglais. Le signal audio est capturé par le microphone, puis traité par l'outil VOSK, un moteur de reconnaissance vocale open source, qui transforme la voix en texte à l'aide d'un modèle entraîné spécifiquement pour la langue anglaise [1]. Une fois le texte obtenu, il est transmis à une interface OpenAI, qui joue le rôle de moteur d'intelligence artificielle pour interpréter la commande, comprendre l'intention de l'utilisateur et générer une réponse adaptée au contexte [12]. Finalement, cette réponse est exploitée pour déclencher l'exécution de l'action correspondante dans le système, bouclant ainsi le cycle complet de l'assistant vocal intelligent.

4.2.3 Analyse des métriques d'entraînement

Les figures 4.4 à 4.11 présentent une analyse complète des métriques d'entraînement collectées durant le développement du modèle de traduction automatique neuronale pour la paire de langues arabe tunisien-anglais. Cette analyse détaillée permet d'évaluer la performance et la stabilité du modèle tout au long du processus d'apprentissage.

Courbes de perte

La progression des pertes d'entraînement et de validation au fil du temps est illustrée dans la Figure 4.4.

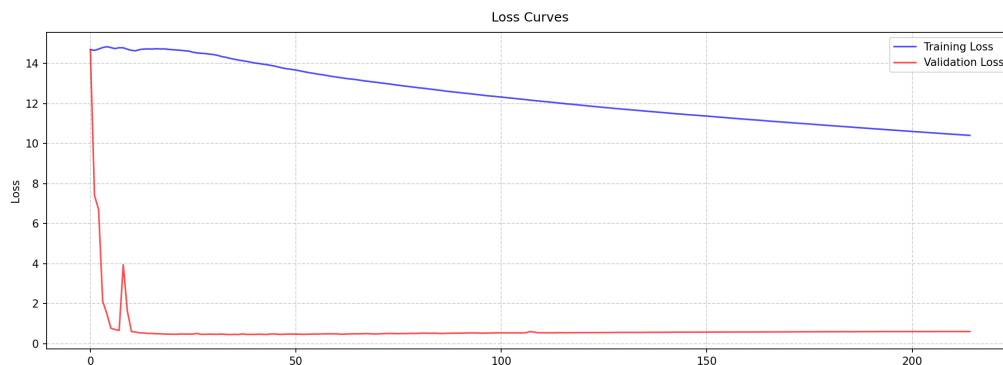


FIGURE 4.4 – Progression des pertes d'entraînement et de validation au fil du temps

Ce graphique suit la progression de la perte d'entraînement (en bleu) et de la perte de validation (en rouge) au cours des étapes d'entraînement. La perte d'entraînement commence autour de 14 et diminue progressivement jusqu'à environ 10, indiquant un apprentissage régulier et cohérent. La perte de validation présente un comportement plus dramatique, commençant élevée, chutant rapidement près de zéro, connaissant un bref pic, puis se stabilisant à une valeur très basse inférieure à 1. Cela suggère que le modèle a rapidement trouvé des paramètres efficaces pour l'ensemble de validation, témoignant d'une adaptation rapide aux données de test.

Calendrier du taux d'apprentissage

La progression du taux d'apprentissage durant l'entraînement est montrée dans la Figure 4.5.

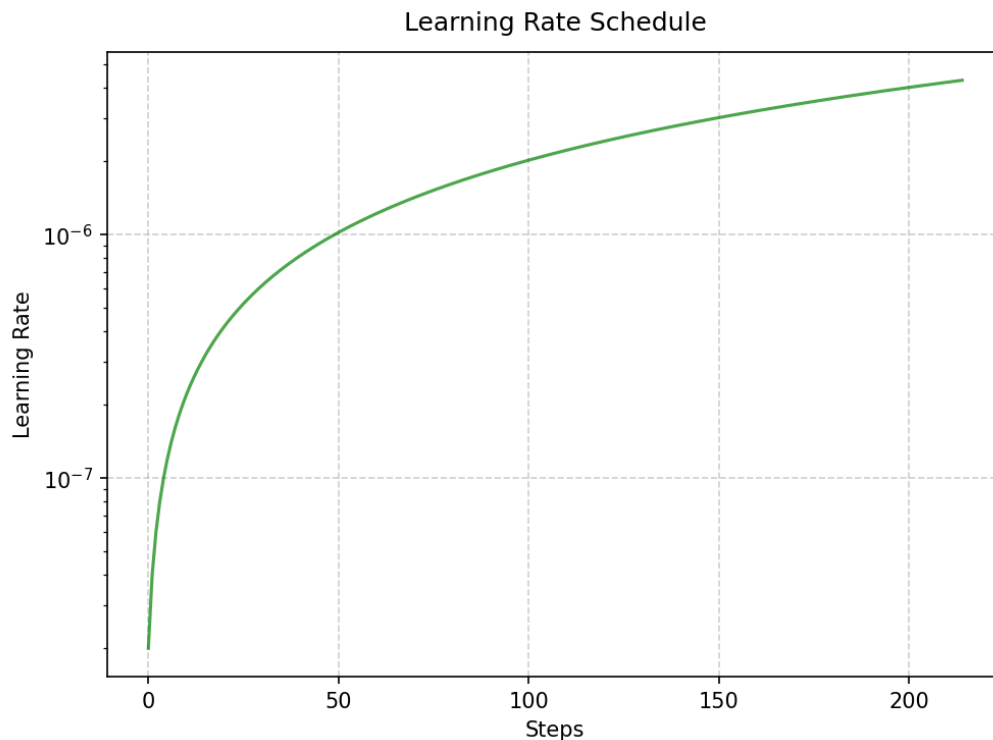


FIGURE 4.5 – Progression du taux d'apprentissage durant l'entraînement

Ce graphique logarithmique montre la progression du taux d'apprentissage tout au long de l'entraînement. Partant d'environ 10^{-8} , le taux augmente progressivement suivant une courbe régulière jusqu'à environ 10^{-4} . Ce calendrier de préchauffage aide à stabiliser l'entraînement initial en commençant par de petites mises à jour de paramètres qui deviennent plus importantes à mesure que l'entraînement progresse. Cette stratégie permet au modèle d'effectuer des ajustements de paramètres plus significatifs une fois la stabilité initiale atteinte, optimisant ainsi le processus d'apprentissage.

Distribution des pertes

L'histogramme des valeurs de perte d'entraînement et de validation est présenté dans la Figure 4.6.

Cet histogramme illustre la distribution des valeurs de perte pour les ensembles d'entraînement (bleu) et de validation (orange). La perte de validation est fortement concentrée près de zéro, avec un pic prononcé indiquant une performance cohérente sur les échantillons de validation. En revanche, la perte d'entraînement montre une

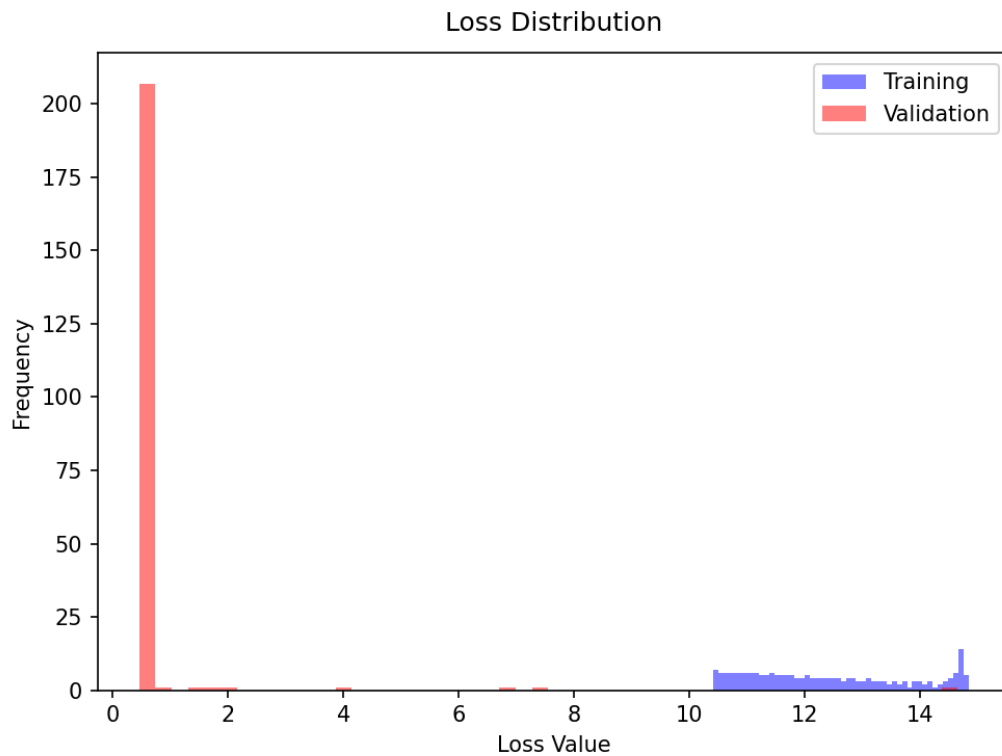


FIGURE 4.6 – Histogramme des valeurs de perte d’entraînement et de validation

distribution plus large à travers des valeurs plus élevées, centrées autour de 12-14, suggérant que le modèle trouve les exemples d’entraînement plus difficiles et plus divers en termes de complexité linguistique et structurelle.

Utilisation de la mémoire au fil du temps

La consommation de mémoire RAM et GPU durant l’entraînement est illustrée dans la Figure 4.7.

Ce graphique suit la consommation de mémoire tout au long de l’entraînement, avec l’utilisation de la RAM (magenta) et la mémoire GPU (cyan) tracées séparément. La mémoire GPU reste constamment élevée à environ 7000 Mo, indiquant que le modèle utilise pleinement la mémoire graphique disponible pour les calculs parallèles nécessaires à l’entraînement des réseaux de neurones. L’utilisation de la RAM fluctue plus significativement entre environ 3500 Mo et 5000 Mo, avec des pics et des chutes périodiques qui correspondent probablement aux événements de traitement par lots et de collecte automatique des déchets mémoire.

Taux de changement de perte

Le taux de changement des valeurs de perte entre étapes consécutives est présenté dans la Figure 4.8.

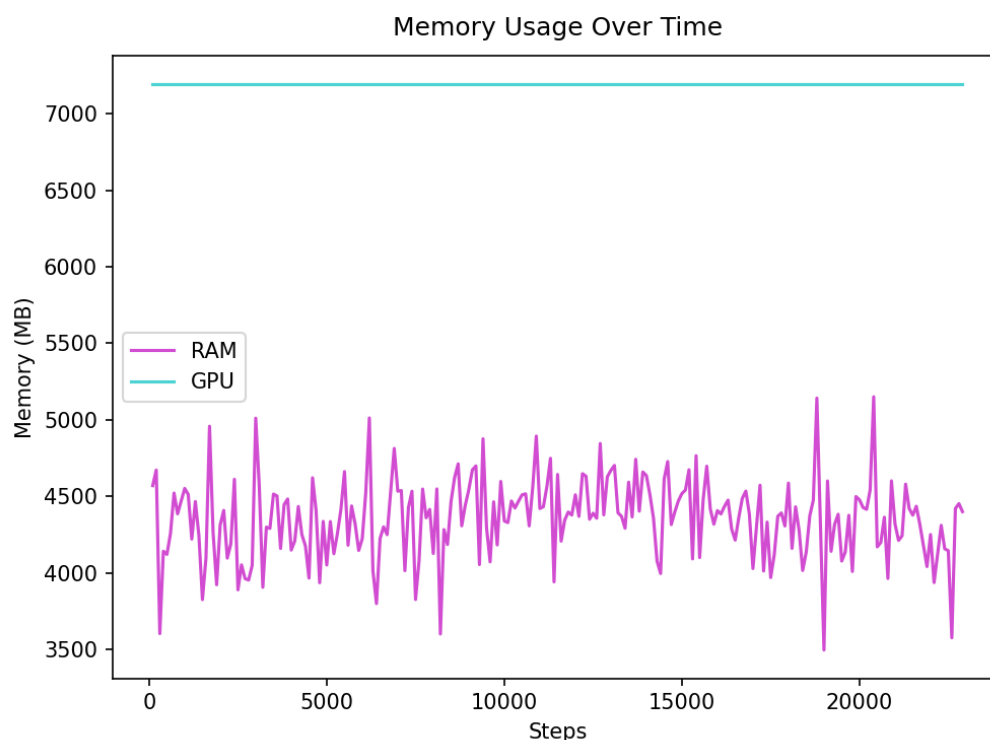


FIGURE 4.7 – Consommation de mémoire RAM et GPU durant l’entraînement

Ce graphique affiche le taux de changement des valeurs de perte entre les étapes d’entraînement consécutives pour la perte d’entraînement (bleu) et la perte de validation (rouge). Après une volatilité initiale avec des changements positifs et négatifs significatifs, particulièrement dans la perte de validation, les deux métriques se stabilisent progressivement à des changements proches de zéro. Cela indique que le modèle a atteint un point relativement stable dans l’espace des paramètres où les mises à jour n’entraînent que des changements minimaux de performance, suggérant une convergence effective vers un optimum local.

Matrice de perte d’entraînement

La visualisation en carte thermique de la progression de la perte d’entraînement est montrée dans la Figure 4.9.

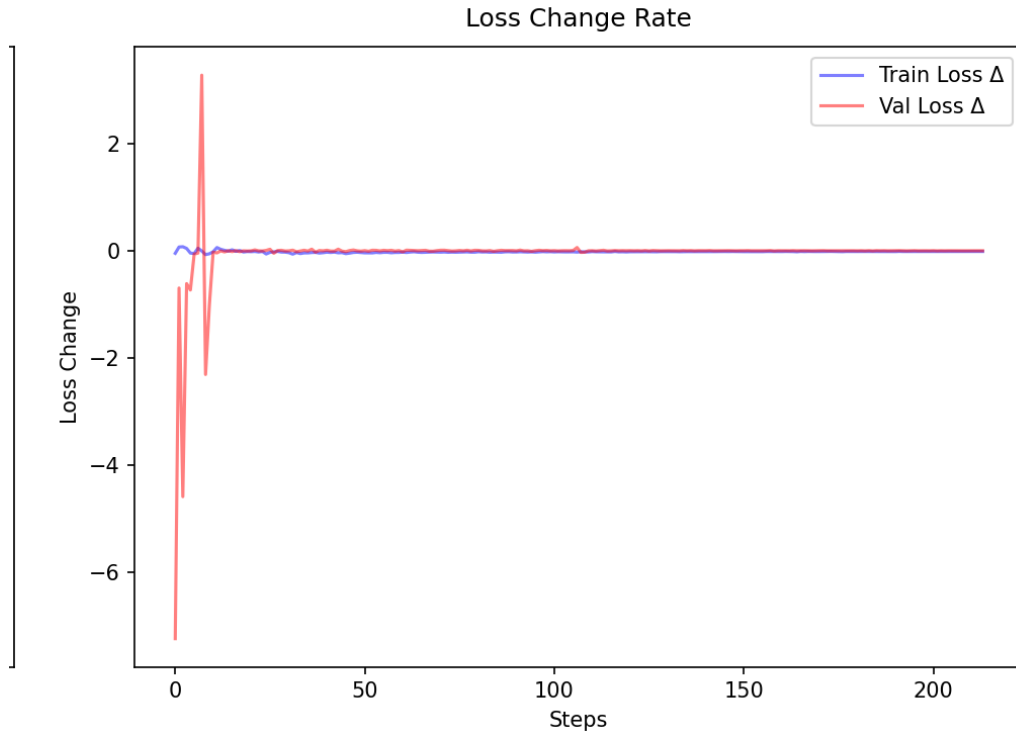


FIGURE 4.8 – Taux de changement des valeurs de perte entre étapes consécutives

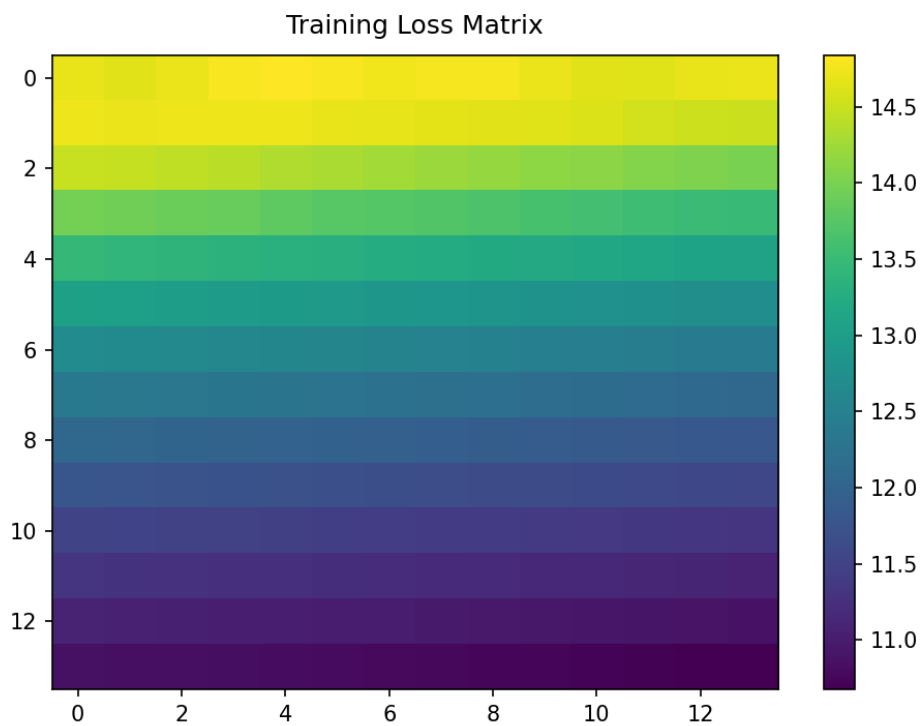


FIGURE 4.9 – Visualisation en carte thermique de la progression de la perte d'entraînement

Cette carte thermique visualise les valeurs de perte d'entraînement disposées sous

forme de matrice, offrant une perspective alternative sur la progression de l'apprentissage. Les couleurs passent du clair, représentant des valeurs de perte plus élevées autour de 14-14,5 en haut, au foncé, indiquant des valeurs de perte plus basses autour de 11-11,5 en bas. Cette transition révèle l'amélioration progressive et constante de la performance d'entraînement. Cette visualisation matricielle aide à identifier les modèles de diminution de la perte tout au long du processus d'entraînement et à détecter d'éventuelles anomalies ou plateaux dans l'apprentissage.

Ratio perte d'entraînement/validation

Le ratio entre perte d'entraînement et perte de validation au fil du temps est présenté dans la Figure 4.10.

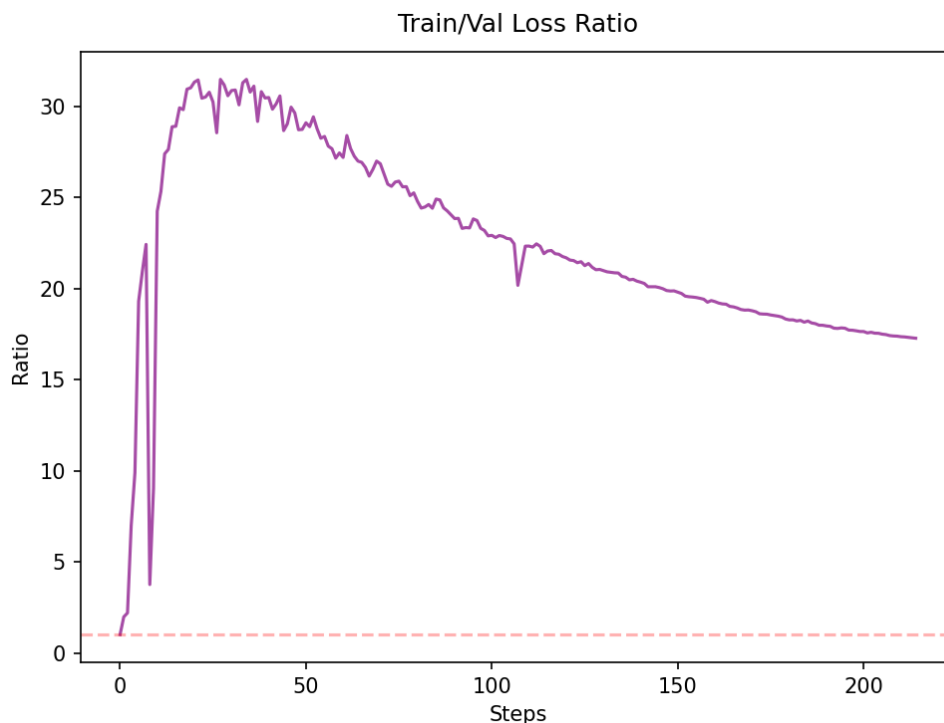


FIGURE 4.10 – Ratio entre perte d'entraînement et perte de validation au fil du temps

Ce graphique trace le ratio entre la perte d'entraînement et la perte de validation au fil du temps. Après une instabilité initiale, le ratio augmente considérablement pour atteindre un pic autour de 30-32, puis diminue progressivement jusqu'à environ 15-17 à la fin de l'entraînement. Ce ratio élevé indique un phénomène de surapprentissage significatif, où le modèle performe beaucoup mieux sur les données de validation que sur les données d'entraînement. Ce comportement est courant dans les tâches de traduction automatique où l'ensemble de validation peut contenir des exemples plus simples, plus standardisés, ou présentant des structures linguistiques plus régulières que l'ensemble d'entraînement.

Moyennes mobiles de perte de validation

Les moyennes mobiles de la perte de validation avec différentes tailles de fenêtre sont illustrées dans la Figure 4.11.

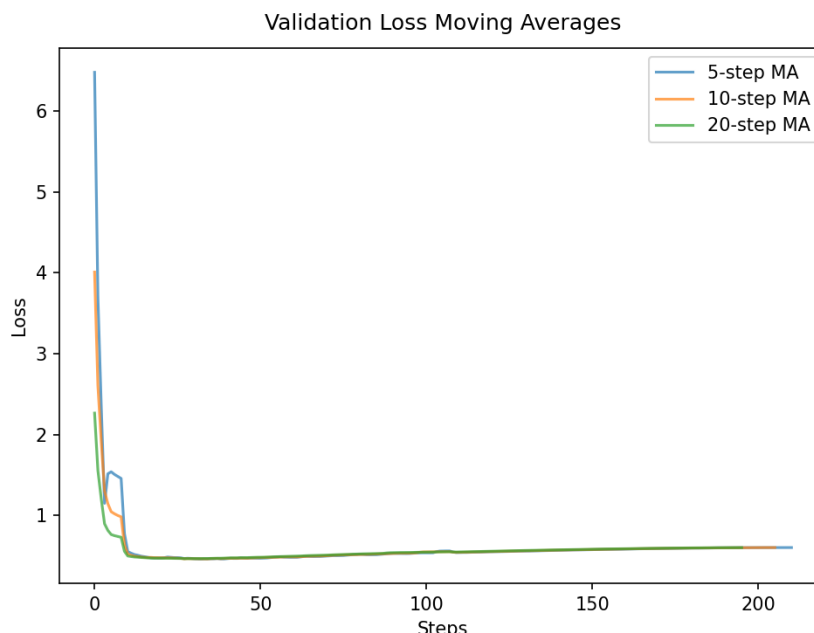


FIGURE 4.11 – Moyennes mobiles de la perte de validation avec différentes tailles de fenêtre

Ce graphique montre diverses moyennes mobiles calculées sur 5 étapes, 10 étapes et 20 étapes de la perte de validation, techniques qui permettent de lisser le bruit statistique et de visualiser les tendances sous-jacentes de l'apprentissage. Après des pics initiaux autour de 6, les trois moyennes convergent rapidement et se stabilisent en dessous de 1, les fenêtres plus longues (20 étapes, ligne verte) montrant la trajectoire la plus lisse et la plus stable. Cette convergence rapide confirme que la performance de validation s'est stabilisée rapidement et est restée cohérente tout au long du processus d'entraînement, témoignant de la robustesse du modèle développé.

Analyse globale des métriques

Les métriques d'entraînement révèlent un modèle de traduction automatique neuronale qui s'est rapidement adapté aux données de validation tout en continuant à trouver les données d'entraînement plus difficiles à traiter. L'écart important entre la perte d'entraînement et de validation suggère un certain degré de surapprentissage, phénomène courant dans les tâches de traduction automatique neuronale. L'utilisation de la mémoire est restée stable et prévisible, indiquant une gestion efficace des ressources computationnelles disponibles. Le calendrier du taux d'apprentissage a mis en œuvre avec succès une stratégie de préchauffage graduel, et le

modèle a convergé vers une performance stable dans le premier quart des étapes d'entraînement, démontrant l'efficacité de l'architecture choisie.

Ces visualisations détaillées ont été créées à partir des métriques collectées durant l'entraînement d'un modèle de traduction automatique neuronale basé sur l'architecture mBART, spécifiquement optimisé pour la tâche complexe de traduction de l'arabe tunisien vers l'anglais. L'ensemble de données utilisé contenait 23037 paires de phrases soigneusement sélectionnées, et l'entraînement a été effectué en utilisant un fractionnement stratégique du jeu de données pour s'adapter aux contraintes de mémoire GPU disponible.

Pour les détails complets d'implémentation et de reproduction, veuillez consulter les fichiers de code associés : `process.py` pour la préparation et le prétraitement des données, ainsi que `train.py` pour le pipeline complet d'entraînement et d'évaluation du modèle.

4.3 Conclusion

Ce chapitre a permis de présenter les fondements théoriques et pratiques de l'intelligence artificielle appliquée à notre système de traduction automatique neuronale. En explorant les sous-domaines de l'IA tels que le Machine Learning, le Deep Learning et la Computer Vision, nous avons mis en lumière les technologies clés mobilisées dans notre projet et leur pertinence pour les défis linguistiques abordés.

Nous avons ensuite détaillé l'architecture complète du pipeline de traitement vocal basé sur l'IA, en particulier l'intégration harmonieuse de la reconnaissance vocale avec VOSK [1], la traduction intermédiaire et le dialogue intelligent via les API d'OpenAI [12] (voir Figure 4.3). L'analyse approfondie des métriques d'entraînement du modèle mBART, présentée à travers les figures 4.4, 4.5, 4.6, 4.7, 4.8, 4.9, 4.10, et 4.11, révèle une progression encourageante avec une convergence rapide et stable, bien que certaines manifestations de surapprentissage aient été observées et documentées.

Ces résultats constituent une base solide et prometteuse pour de futures améliorations, notamment en termes de techniques de régularisation avancées, d'enrichissement du corpus d'entraînement avec des données plus diversifiées, et d'adaptation fine au contexte linguistique et culturel tunisien. L'architecture développée démontre la faisabilité technique de notre approche et ouvre la voie à des développements futurs ambitieux dans le domaine de la traduction automatique pour les dialectes arabes.

Chapitre 5

Sprint - 4 : Design 3D & Interface mobile

Table des matières

5.1	Introduction	58
5.2	Conception 3D du prototype	58
5.2.1	Présentation de Blender	58
5.2.2	Design du robot BEEMO	59
5.2.3	Processus de conception	60
5.2.4	Composants modélisés	60
5.2.5	Caractéristiques techniques du boîtier	61
5.2.6	Optimisations pour l'impression 3D	61
5.3	Développement de l'interface mobile	62
5.3.1	Flutter : Framework de développement	62
5.3.2	Architecture de l'application	62
5.3.3	Fonctionnalités principales	63
5.3.4	Design UX/UI	63
5.3.5	Sécurisation de l'application	70
5.3.6	Gestion des appareils et des lieux	70
5.3.7	Tests et validation	70
5.4	Conclusion	71

5.1 Introduction

Ce chapitre présente les étapes finales de développement de notre assistant robotisé BEEMO, en se concentrant sur deux aspects essentiels : la conception 3D du boîtier et le développement de l'interface mobile. Ces composants sont cruciaux pour créer un produit fini qui soit à la fois fonctionnel et attrayant pour l'utilisateur final. Voici

les grandes lignes de ce chapitre :

- **Conception 3D** : modélisation du boîtier avec Blender, intégration des composants, solutions de ventilation et fixation.
- **Développement mobile** : création d'une interface utilisateur avec Flutter, implémentation du tableau de bord en temps réel, contrôle manuel et affichage des données capteurs.
- **Design UX/UI** : création d'une expérience utilisateur intuitive et d'une interface graphique attrayante.
- **Sécurisation** : implémentation des mesures de sécurité pour protéger l'accès à l'application et aux données.

5.2 Conception 3D du prototype

La conception du boîtier du robot BEEMO a nécessité l'utilisation d'outils de modélisation 3D performants pour assurer une intégration parfaite des composants électroniques tout en garantissant une esthétique soignée. Pour ce projet, nous avons opté pour Blender, un logiciel de modélisation 3D professionnel et open source.

5.2.1 Présentation de Blender

Le logo de Blender est illustré dans la Figure 5.1.



FIGURE 5.1 – Logo de Blender

Blender est un logiciel libre et gratuit de modélisation, animation et rendu 3D. Contrairement à ce qui était initialement prévu (Fusion 360), nous avons choisi Blender pour plusieurs raisons :

- **Polyvalence** : Blender offre un ensemble complet d'outils pour la modélisation 3D, la sculpture, l'animation, la simulation, le rendu, la composition et le suivi de mouvement.
- **Communauté active** : La communauté mondiale de Blender fournit des ressources, tutoriels et bibliothèques de modèles qui ont accéléré notre processus de développement.
- **Compatibilité** : Blender prend en charge l'importation et l'exportation de nombreux formats de fichiers 3D, ce qui facilite l'intégration avec les logiciels d'impression 3D.
- **Modélisation paramétrique** : Grâce aux add-ons comme "CAD Sketcher", nous avons pu réaliser une modélisation précise pour les pièces mécaniques du robot.
- **Gratuité et licence open source** : En accord avec notre philosophie de projet, l'utilisation d'un logiciel libre nous a permis de maîtriser les coûts tout en conservant une liberté totale sur nos créations.

5.2.2 Design du robot BEEMO

Le design du robot BEEMO s'inspire des tendances actuelles en robotique domestique, avec une approche minimaliste et épurée. Le robot présente les caractéristiques suivantes :

- **Silhouette conviviale** : Formes douces et arrondies pour une apparence non-intimidante et sympathique.
- **Design bicolore** : Combinaison de blanc et de bleu pour une esthétique moderne et apaisante.
- **Écran frontal** : Interface visuelle principale intégrée dans une face noire contrastant avec l'ensemble.
- **Base stable** : Socle circulaire bleu permettant une stabilité optimale et une rotation fluide.
- **Membres stylisés** : Bras simplifiés qui contribuent à l'apparence amicale du robot.

Les différentes vues du robot BEEMO sont présentées dans la Figure 5.2.



FIGURE 5.2 – Différentes vues du robot BEEMO

5.2.3 Processus de conception

La création du boîtier a suivi une méthodologie rigoureuse, comprenant plusieurs étapes clés :

1. **Étude préliminaire** : Analyse des dimensions et des contraintes liées aux composants électroniques (Raspberry Pi, écran, servomoteurs, etc.).
2. **Esquisse conceptuelle** : Réalisation de croquis initiaux pour définir l'esthétique générale du robot.
3. **Modélisation 3D** : Création du modèle numérique complet avec Blender, en tenant compte des contraintes d'assemblage.
4. **Test de compatibilité** : Vérification numérique de l'intégration de tous les composants.
5. **Optimisation** : Amélioration du design pour faciliter l'impression 3D et l'assemblage.
6. **Prototypage** : Impression des pièces et assemblage du prototype physique.

5.2.4 Composants modélisés

Plusieurs pièces principales ont été conçues pour constituer l'enveloppe externe du robot BEEMO :

- **Tête rotative** : Abritant l'écran et le système de caméra, avec un degré de liberté pour permettre la rotation.
 - **Corps principal** : Contenant la carte Raspberry Pi, les capteurs et la batterie.
 - **Articulations mobiles** : Permettant un mouvement naturel des bras.
 - **Base circulaire** : Assurant la stabilité et contenant les moteurs pour la rotation.
-

5.2.5 Caractéristiques techniques du boîtier

Le boîtier conçu présente plusieurs caractéristiques techniques importantes :

- **Dimensions globales** : 120mm × 150mm × 200mm (largeur × profondeur × hauteur).
- **Système de ventilation** : Grilles d'aération discrètes sur les côtés pour dissiper la chaleur générée par le Raspberry Pi.
- **Points de fixation** : Emplacements pour vis et entretoises permettant un assemblage solide et démontable.
- **Gestion des câbles** : Passages intégrés pour organiser le câblage interne.
- **Matériau recommandé** : PLA ou PETG pour l'impression 3D (résistance et facilité d'impression).
- **Épaisseur des parois** : 2mm pour un compromis optimal entre solidité et économie de matière.
- **Compatibilité** : Conçu pour s'adapter parfaitement aux composants électroniques spécifiés dans le chapitre précédent.

5.2.6 Optimisations pour l'impression 3D

Plusieurs optimisations ont été apportées au design pour garantir une impression 3D réussie :

- **Élimination des surplombs critiques** : Les angles supérieurs à 45° ont été évités ou supportés.
- **Division en modules** : Le boîtier a été conçu en plusieurs parties pour faciliter l'impression sans support.
- **Chanfreins et arrondis** : Ajout de chanfreins et d'arrondis pour renforcer les zones vulnérables et améliorer l'esthétique.
- **Tolérance d'assemblage** : Prise en compte d'un jeu de 0.2mm entre les pièces s'emboîtant pour compenser les imprécisions d'impression.
- **Rainures d'auto-alignement** : Intégration de guides pour faciliter l'assemblage des différentes parties.

Cette conception 3D soignée a permis d'obtenir un boîtier à la fois fonctionnel, esthétique et parfaitement adapté aux besoins du projet BEEMO.

5.3 Développement de l'interface mobile

L'interface mobile constitue un élément essentiel du projet BEEMO, permettant à l'utilisateur de contrôler le robot et de visualiser les données capteurs à distance. Cette section détaille le processus de développement de l'application mobile, depuis sa conception jusqu'à son implémentation finale.

5.3.1 Flutter : Framework de développement

Le logo de Flutter est affiché dans la Figure 5.3.



FIGURE 5.3 – Logo de Flutter

Flutter est un framework open source développé par Google pour la création d'applications multiplateformes. Nous avons choisi Flutter pour plusieurs raisons :

- **Multiplateforme** : Déploiement sur Android et iOS à partir d'une base de code unique, réduisant le temps de développement.
- **Performances natives** : Contrairement à d'autres frameworks hybrides, Flutter compile directement en code natif, offrant des performances équivalentes aux applications natives.
- **Interface graphique riche** : Large bibliothèque de widgets personnalisables permettant de créer des interfaces utilisateur élégantes et réactives.
- **Hot Reload** : Fonctionnalité permettant de voir immédiatement les modifications apportées au code, accélérant considérablement le développement.
- **Écosystème mature** : Nombreux packages disponibles pour l'intégration de fonctionnalités comme la connectivité Bluetooth, le stockage local, ou l'authentification.

5.3.2 Architecture de l'application

L'application BEEMO suit une architecture MVVM (Model-View-ViewModel) pour séparer clairement la logique métier de l'interface utilisateur. Cette approche facilite la maintenance et les tests unitaires.

- **Model** : Représente les données et la logique métier, incluant les classes pour les capteurs, les commandes et l'état du robot.
 - **View** : Composée des écrans et widgets Flutter, responsable uniquement de l'affichage des données et de la capture des interactions utilisateur.
-

-
- **ViewModel** : Sert d'intermédiaire entre le Model et le View, transformant les données du modèle en un format adapté à l'affichage et traitant les actions utilisateur.
 - **Services** : Couche supplémentaire gérant la communication avec le robot (Bluetooth, Wi-Fi), le stockage local et les API externes.

5.3.3 Fonctionnalités principales

L'application mobile BEEMO offre plusieurs fonctionnalités clés pour contrôler et surveiller le robot assistant :

1. **Dashboard en temps réel** : Affichage des données météo et environnementales (température, humidité, pression atmosphérique) ainsi que de l'état du robot.
2. **Contrôle des appareils domotiques** : Interface permettant de gérer les appareils connectés par pièce (lumières, climatisation, sécurité, chauffe-eau).
3. **Gestion des automatisations** : Configuration des règles et scénarios d'automatisation avec planification temporelle.
4. **Paramètres utilisateur** : Personnalisation de l'interface et des préférences de notification.
5. **Connexion directe au robot** : Bouton dédié pour établir une connexion Bluetooth avec BEEMO.

5.3.4 Design UX/UI

Une attention particulière a été portée à l'expérience utilisateur (UX) et à l'interface graphique (UI) pour garantir une utilisation intuitive et agréable.

Ces images montrent une interface utilisateur intuitive pour interagir avec le robot BEEMO et gérer un environnement domestique connecté.

Rôle de l'Interface Écran Mobile

L'écran de l'application mobile BEEMO agit comme le point central d'interaction et de visualisation pour l'utilisateur. Son rôle principal est de fournir une interface graphique complète permettant de :

- **Connecter et configurer le robot** : L'utilisateur peut établir une connexion avec le robot BEEMO via un identifiant unique et accéder à ses paramètres de fonctionnement.
 - **Visualiser les données en temps réel** : Le tableau de bord affiche des informations cruciales telles que les données météorologiques, l'état opérationnel du robot, et les données des capteurs connectés.
-

-
- **Contrôler les appareils connectés** : L'interface offre un moyen de gérer manuellement les dispositifs domotiques (lumières, climatisation, etc.) et de configurer des automatisations.
 - **Interagir avec l'assistant IA** : Un module de chat dédié permet à l'utilisateur de communiquer avec l'intelligence artificielle du robot via des commandes textuelles, recevant des réponses et des informations.

En somme, l'écran sert de hub visuel pour une gestion complète et intuitive de l'écosystème BEEMO, rendant l'interaction homme-robot fluide et informative.

Écran Principal du Tableau de Bord (Dashboard)

L'écran principal du Tableau de Bord (Dashboard) est illustré dans la Figure 5.4.

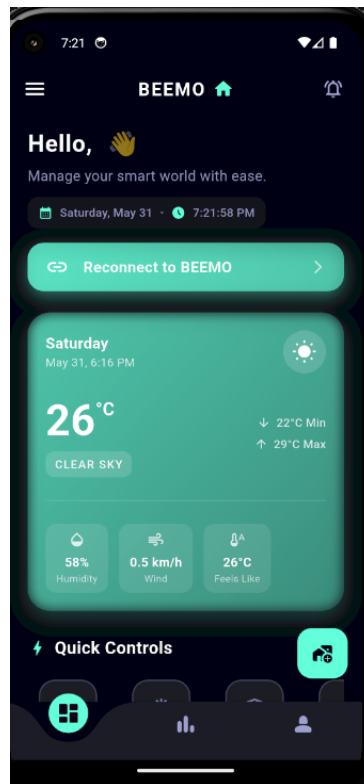


FIGURE 5.4 – Écran principal du Tableau de Bord (Dashboard)

Cette interface est le tableau de bord principal et d'accueil de l'application BEEMO. Son rôle est de fournir à l'utilisateur une vue d'ensemble rapide et des points d'accès essentiels à son écosystème intelligent, avec une emphase sur l'information environnementale et l'état du système.

- **En-tête** : Contient le nom de l'application, une icône de maison, ainsi que les icônes de menu (trois barres horizontales) et de notifications (cloche).
 - **Message de bienvenue** : Affichage de « Hello » avec une icône de main, suivi du slogan *"Manage your smart world with ease"*.
-

-
- **Date et heure actuelles** : Présentées dans une barre dédiée pour un suivi en temps réel.
 - **Bouton de reconnexion** : Un bouton vert « Reconnect to BEEMO » permet de rétablir la connexion avec le robot.
 - **Widget météo** : Affiche les informations météorologiques détaillées : température actuelle, état du ciel, températures min/max, humidité, vitesse du vent et température ressentie.
 - **Contrôles rapides (Quick Controls)** : Fournit des raccourcis vers des actions fréquentes pour une interaction simplifiée.
 - **Barre de navigation inférieure** : Permet d'accéder facilement aux principales sections de l'application (tableau de bord, appareils, paramètres, etc.).

Fenêtre de Connexion au Robot

La fenêtre de Connexion au Robot est présentée dans la Figure 5.5.

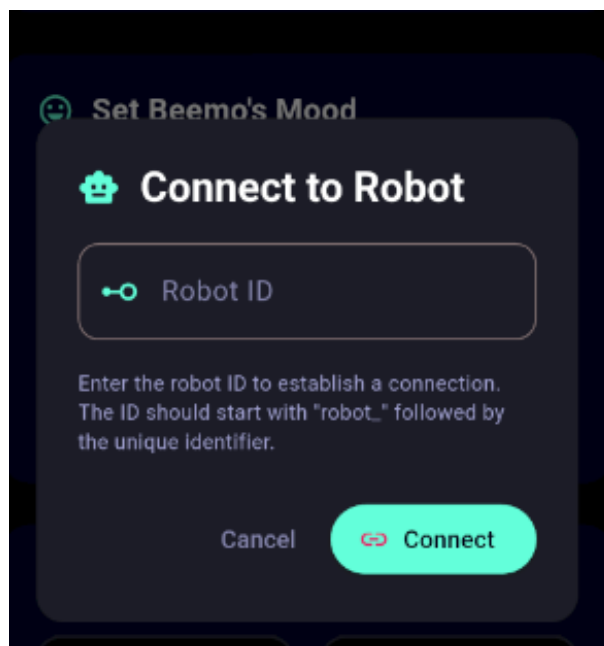


FIGURE 5.5 – Fenêtre de Connexion au Robot

Cette interface est un dialogue crucial pour établir la communication entre l'application et un robot BEEMO spécifique. Elle exige que l'utilisateur saisisse l'ID unique de son robot, garantissant une connexion ciblée.

- **Titre** : Le titre de la boîte de dialogue est "Connect to Robot" (Se connecter au robot).
 - **Champ "Robot ID"** : Un champ de saisie permet à l'utilisateur d'entrer l'identifiant unique du robot ("Robot ID").
-

-
- **Instructions** : Une note explique le format de l’ID : "Enter the robot ID to establish a connection. The ID should start with 'robot_' followed by the unique identifier" (Entrez l’ID du robot pour établir une connexion. L’ID doit commencer par 'robot_' suivi de l’identifiant unique).
 - **Boutons d’Action** : Deux boutons sont disponibles : "Cancel" (Annuler) pour fermer la fenêtre et "Connect" (Connecter) pour lancer la connexion.

Son rôle principal est d’agir comme la passerelle d’authentification et d’identification. Sans cette étape, l’application ne pourrait pas savoir quel robot contrôler ou d’où recevoir des informations. Elle offre un champ de saisie clair, avec des instructions sur le format de l’ID, et des boutons pour initier la connexion ou l’annuler. Essentielle pour la première utilisation ou pour reconnecter un robot, elle assure que seul le robot désigné par l’utilisateur est lié à l’application, sécurisant ainsi les interactions et les commandes.

Écran d’État et de Configuration du Robot

L’écran d’État et de Configuration du Robot est visible dans la Figure 5.6.

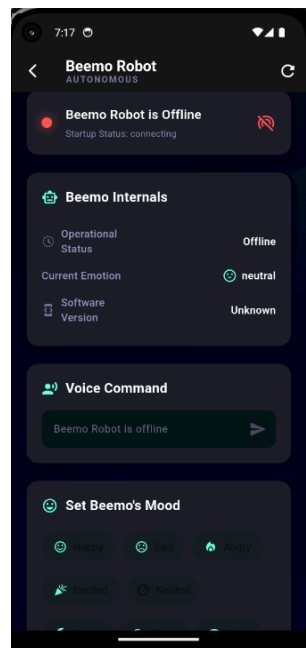


FIGURE 5.6 – Écran d’État et de Configuration du Robot

Le rôle de cette interface de l’application BEEMO est de gérer et contrôler directement le robot BEEMO.

- **Statut de Connexion** : Le premier bloc en haut indique "Beemo Robot is Offline" (Le robot Beemo est hors ligne) avec un statut de démarrage "Startup Status : connecting" (Statut de démarrage : connexion en cours). Une icône à droite semble indiquer une tentative de rafraîchissement ou de reconnexion.
-

-
- **Internes du Robot (Beemo Internals)** : Ce bloc affiche des informations sur l'état interne du robot : Statut Opérationnel : "Offline". Émotion Actuelle : "neutral" avec une icône de visage neutre. Version du Logiciel : "Unknown" (Inconnue).
 - **Commande Vocale (Voice Command)** : Un champ de texte permet à l'utilisateur d'envoyer des commandes vocales au robot, avec une indication "Beemo Robot is offline". Un bouton d'envoi (icône d'avion en papier) est présent.
 - **Définir l'Humeur du Robot (Set Beemo's Mood)** : Le dernier bloc propose des options pour définir l'état émotionnel du robot, avec des icônes et des libellés comme "Happy", "Sad", "Angry", "Excited", "Neutral".

Elle permet de surveiller sa connexion et son état interne. Elle permet d'envoyer des commandes vocales au robot pour qu'il agisse sur les appareils domestiques. Elle permet de définir les émotions du robot. Elle permet d'effectuer des opérations système (redémarrage, arrêt). C'est l'interface principale pour s'assurer que le robot est opérationnel et prêt à exécuter les ordres.

Interface de Gestion des Pièces et des Appareils ("Sala")

L'interface de Gestion des Pièces et des Appareils ("Sala") est illustrée dans la Figure 5.7.

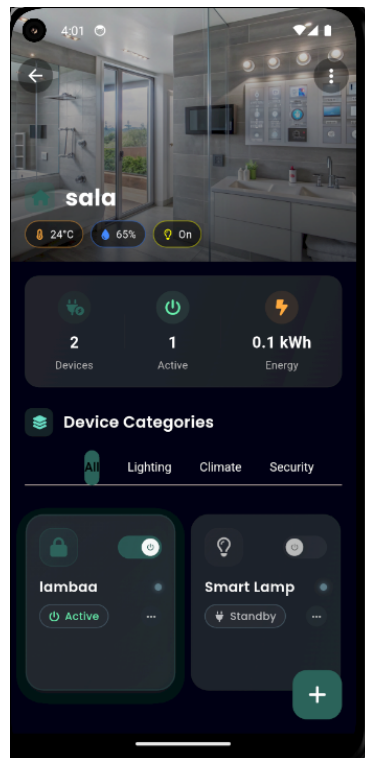


FIGURE 5.7 – Interface : Gestion des Pièces et des Appareils ("Sala")

Cette interface est le tableau de bord de la maison intelligente, permettant à l'uti-

lisateur de :

- **Visualiser et obtenir des statistiques** sur chaque pièce (température, humidité, appareils).
- **Contrôler directement les appareils** par catégorie (lumières, climat, sécurité) et voir leur statut.
- **Ajouter de nouveaux appareils.**

Son rôle est d'offrir un contrôle intuitif et une vue d'ensemble de l'environnement domestique.

Écran de Chat/Assistant (PFE Platform Assistant)

L'écran de Chat/Assistant (PFE Platform Assistant) est présenté dans la Figure 5.8.

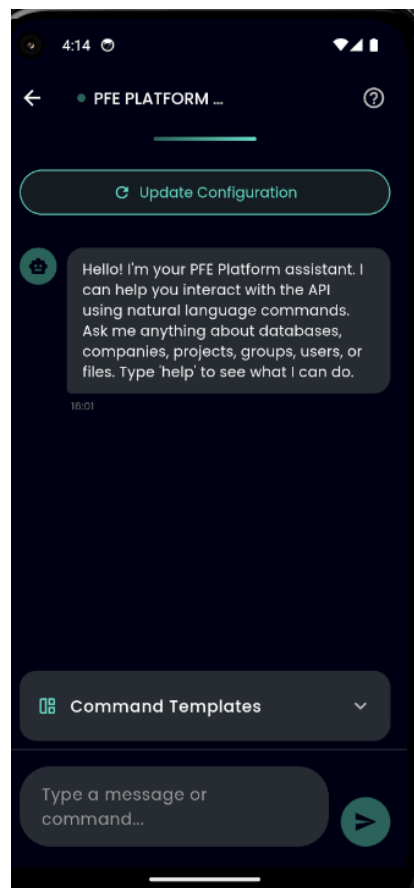


FIGURE 5.8 – Écran de Chat/Assistant (PFE Platform Assistant)

Cette interface a pour rôle central de servir de point d'interaction conversationnel principal entre l'utilisateur et l'API de la plateforme PFE, via un assistant IA. C'est un outil de communication en langage naturel qui démocratise l'accès aux fonctionnalités complexes de l'API.

-
- **En-tête** : Affiche "PFE PLATFORM ..." avec une flèche de retour, un indicateur de statut (point vert) et une icône d'aide (?).
 - **Bouton de Mise à Jour de la Configuration** : Un bouton "Update Configuration" est visible, suggérant la possibilité de recharger ou d'appliquer de nouveaux paramètres.
 - **Message de l'Assistant** : Une bulle de dialogue contient un message de bienvenue de l'assistant : "Hello! I'm your PFE Platform assistant. I can help you interact with the API using natural language commands. Ask me anything about databases, companies, projects, groups, users, or files. Type 'help' to see what I can do." (Bonjour! Je suis votre assistant de plateforme PFE. Je peux vous aider à interagir avec l'API en utilisant des commandes en langage naturel. Posez-moi des questions sur les bases de données, les entreprises, les projets, les groupes, les utilisateurs ou les fichiers. Tapez 'help' pour voir ce que je peux faire.).
 - **Modèles de Commande (Command Templates)** : Une section déroulante "Command Templates" est présente, probablement pour fournir des exemples ou des raccourcis de commandes.
 - **Champ de Saisie** : Un champ de texte en bas permet à l'utilisateur de "Type a message or command..." (Taper un message ou une commande...) avec un bouton d'envoi.

Principes de design appliqués

Pour créer une expérience utilisateur optimale, plusieurs principes de design ont été appliqués :

- **Mode sombre natif** : Application conçue avec un thème sombre par défaut pour une meilleure lisibilité et une consommation d'énergie réduite.
 - **Palette de couleurs limitée** : Utilisation d'une palette restreinte avec le turquoise comme couleur principale et des accents colorés (rouge, orange, vert) pour la signalétique.
 - **Navigation intuitive** : Structure hiérarchique claire avec une barre de navigation inférieure pour accéder rapidement aux fonctionnalités principales.
 - **Feedback visuel** : Animations subtiles et retours visuels pour confirmer les actions de l'utilisateur.
 - **Iconographie consistante** : Système d'icônes unifié pour améliorer la compréhension et la mémorisation des fonctionnalités.
-

5.3.5 Sécurisation de l'application

La sécurité étant une préoccupation majeure, plusieurs mesures ont été mises en place pour protéger l'application et les données utilisateur :

- **Authentification sécurisée** : Système d'authentification avec adresse email et mot de passe, visible sur l'écran des paramètres utilisateur.
- **Chiffrement des données** : Toutes les communications entre l'application et le robot sont chiffrées à l'aide du protocole TLS.
- **Stockage sécurisé** : Les informations sensibles sont stockées localement dans un espace sécurisé (Keychain sur iOS, KeyStore sur Android).
- **Contrôle d'accès** : Mise en place de mécanismes de vérification des permissions pour limiter l'accès aux fonctionnalités critiques.
- **Session timeout** : Déconnexion automatique après une période d'inactivité pour prévenir l'accès non autorisé.

5.3.6 Gestion des appareils et des lieux

L'application permet une gestion organisée des appareils connectés par lieu :

- **Organisation par pièces** : Les dispositifs sont regroupés selon leur emplacement dans la maison (salon, bureau, salle de bain, etc.).
- **Ajout manuel de dispositifs** : Interface dédiée permettant d'ajouter des appareils compatibles en spécifiant leur nom et identifiant.
- **État de connexion visible** : Indicateur d'état montrant clairement si l'appareil est connecté et fonctionnel.
- **Contrôle individuel** : Possibilité d'activer/désactiver individuellement chaque appareil depuis son panneau dédié.

5.3.7 Tests et validation

Pour garantir la qualité et la fiabilité de l'application, plusieurs types de tests ont été effectués :

1. **Tests unitaires** : Vérification du bon fonctionnement de chaque composant isolé.
 2. **Tests d'intégration** : Validation de l'interaction entre les différents modules de l'application.
 3. **Tests d'interface utilisateur** : Automatisation des interactions utilisateur pour vérifier le comportement de l'interface.
 4. **Tests de compatibilité** : Vérification du fonctionnement sur différents appareils et versions d'Android/iOS.
-

5. **Tests de performance** : Mesure des temps de réponse et de la consommation de ressources (mémoire, batterie).

L'application mobile BEEMO constitue une interface complète et intuitive pour interagir avec le robot assistant. Son développement avec Flutter a permis d'obtenir une solution multiplateforme performante, tout en garantissant une expérience utilisateur de qualité et un niveau de sécurité élevé.

5.4 Conclusion

Ce sprint a permis de finaliser les aspects cruciaux du projet BEEMO en livrant à la fois une conception 3D aboutie et une interface mobile fonctionnelle. Ces deux composants représentent la partie visible du projet pour l'utilisateur final et contribuent grandement à l'expérience globale. La conception 3D avec Blender a donné naissance à un robot au design moderne et convivial, optimisé pour l'impression 3D et l'intégration des composants électroniques. Le choix d'un design bicolore blanc et bleu, associé à des lignes douces et arrondies, confère au robot BEEMO une apparence amicale qui s'intègre harmonieusement dans un environnement domestique (voir Figure 5.2). Parallèlement, le développement de l'application mobile avec Flutter offre une interface utilisateur intuitive et performante, accessible sur les principales plateformes mobiles. L'architecture MVVM adoptée garantit une maintenance facilitée et une évolutivité du code, tandis que la conception UI/UX en mode sombre avec une palette de couleurs limitée assure une expérience utilisateur agréable et cohérente. Les fonctionnalités implémentées dans l'interface mobile permettent non seulement de contrôler le robot BEEMO, mais également de gérer un écosystème domotique complet, avec la possibilité d'organiser les appareils par pièce et de créer des automatisations personnalisées. L'attention particulière portée à la sécurité, tant au niveau de l'authentification que du stockage et de la transmission des données, répond aux exigences actuelles en matière de protection de la vie privée et de cybersécurité.

Conclusion Générale

Le projet **BEEMO robot**, mené selon la méthode agile Scrum, a permis de structurer de manière progressive et itérative le développement d'un assistant robotisé interactif, combinant intelligence artificielle, interface mobile et capteurs embarqués. Chaque Sprint a contribué à construire les fondations solides du système en s'attaquant à des aspects clés du projet.

Le **Sprint 1** a posé l'architecture du système à travers une modélisation UML complète, clarifiant les interactions entre l'utilisateur, le robot et l'application mobile, ainsi que la structuration des composants logiciels. Ce travail initial a été essentiel pour cadrer les développements futurs.

Le **Sprint 2** s'est concentré sur la mise en place du système IoT, tant sur le plan matériel que logiciel. L'intégration des capteurs, du module de reconnaissance vocale locale (Vosk), des moteurs et de la communication via API REST ou MQTT a permis de rendre possible une première interaction vocale basique entre l'utilisateur et le robot.

Le **Sprint 3** a introduit une couche d'intelligence artificielle, en intégrant un modèle distant pour la compréhension du langage naturel, permettant au robot d'interpréter des commandes complexes. Ce sprint a aussi jeté les bases d'une gestion des données structurée et sécurisée entre les différents composants du système.

Enfin, le **Sprint 4** a permis de finaliser l'enveloppe matérielle et l'expérience utilisateur grâce à la modélisation 3D du robot et au développement d'une interface mobile en Flutter, à la fois ergonomique, sécurisée et réactive.

Ces quatre Sprints ont permis d'aboutir à une version fonctionnelle du **BEEMO robot**, un assistant intelligent capable d'interagir naturellement avec l'utilisateur et de contrôler son environnement.

Perspectives

Les fondations solides établies par ce projet ouvrent la voie à de nombreuses évolutions futures pour le BEEMO robot. Parmi les perspectives envisagées, nous pouvons citer :

- **Intégration de nouvelles fonctionnalités** : Enrichir l'assistant avec des capacités supplémentaires, telles que la gestion d'un agenda personnel, l'accès à des informations en temps réel (actualités, trafic), ou l'intégration avec d'autres services tiers.
 - **Apprentissage automatique avancé** : Développer des modèles d'appren-
-

tissage automatique pour permettre au robot de s'adapter aux habitudes de l'utilisateur, d'anticiper ses besoins et de personnaliser ses réponses de manière plus proactive.

- **Extension de la domotique contrôlée** : Élargir la compatibilité du BEEMO robot avec un plus grand nombre d'appareils domotiques et de protocoles de communication (Zigbee, Z-Wave) pour offrir un contrôle plus étendu de l'environnement.
- **Amélioration de l'interaction physique** : Explorer des mouvements plus complexes et expressifs pour les servomoteurs afin de rendre l'interaction encore plus naturelle et engageante.
- **Optimisation des performances locales** : Continuer à optimiser les algorithmes de traitement vocal et d'IA pour minimiser la latence et réduire la consommation de ressources, renforçant ainsi l'autonomie du système.
- **Déploiement en environnement réel** : Tester le prototype dans des scénarios d'utilisation quotidiens pour recueillir les retours des utilisateurs et affiner le système en fonction des besoins réels.
- **Limite matérielle du modèle AI tunisien** : Un modèle IA personnalisé a été développé pour comprendre la langue tunisienne. Cependant, ce modèle nécessite une puissance de calcul supérieure à celle qu'offre la carte Raspberry Pi actuellement utilisée. Pour garantir son bon fonctionnement, une migration vers une carte plus performante (comme une Jetson Nano ou un mini PC) est envisagée. Ce besoin est illustré dans la Figure 5.9.

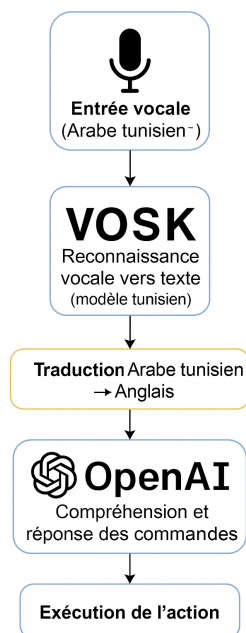


FIGURE 5.9 – Contraintes matérielles pour l'exécution locale du modèle AI tunisien.

Bibliographie

- [1] Alpha Cephei Ltd. Vosk : Offline open source speech recognition toolkit, 2024.
- [2] Christopher M. Bishop.
- [3] Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley Professional, 1999.
- [4] DeepSeek. Deepseek ai api documentation.
- [5] Roy T. Fielding. Architectural styles and the design of network-based software architectures. *Ph.D. dissertation, University of California, Irvine*, 2000.
- [6] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [7] Google LLC. Flutter documentation, 2024.
- [8] Diodes Incorporated. Technical report.
- [9] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11) :2278–2324, 1998.
- [10] OASIS Standard. Mqtt version 5.0, 2019. Consulté le 31 mai 2025.
- [11] Object Management Group (OMG). Omg unified modeling language (omg uml) specification, 2017.
- [12] OpenAI. Openai api documentation, 2024.
- [13] Raspberry Pi Foundation. Raspberry pi documentation, 2024.
- [14] Stuart J. Russell and Peter Norvig. *Artificial Intelligence : A Modern Approach*. Pearson, 2020.
- [15] Scrum.org & Scrum Inc. Le guide scrum : Le guide officiel de scrum, 2020. Consulté le 31 mai 2025.
- [16] Richard Szeliski. *Computer Vision : Algorithms and Applications*. Springer, 2010.
- [17] Abhishek Verma and Shashi Prakash. *Internet of Things (IoT) Fundamentals*. CRC Press, 2020.

MABROUK AYMEN, BEN RAYANA YASSINE

RAPPORT DE SIMILITUDE

0%

INDICE DE SIMILITUDE

0%

SOURCES INTERNET

%

PUBLICATIONS

%

COPIES DE L'ÉTUDIANT

SOURCES PRINCIPALES

Exclure les citations Activer
Exclure la bibliographie Activer

Exclure les correspondances

< 2%
